Euromed University of Fez — École d'Ingénierie Digitale et d'Intelligence Artificielle (EIDIA)
En partenariat avec : Université Sidi Mohamed Ben Abdallah
Faculté de Médecine, de Pharmacie et de Dentisterie de Fès — HAIM

Final Year Project (PFE) Report

AI NéphroCare: An Intelligent Clinical Decision Support Platform for One-Year Mortality Prediction in Hemodialysis Patients

Presented by:
Nisrine Benbellaj

Academic Supervisor:
Ms. Sara Bakkali

Technical Supervisor:
Mr. Zakaria Alami

Academic year:	2025–2026
Program:	Big Data Analytics — EIDIA
Host center:	HAIM, CHU Hassan II Fès

Dedication

To my parents, whose unconditional love and sacrifices
have been the foundation of every step I take.

To my family and friends,
for their endless support and encouragement.

To all patients suffering from kidney disease—
may this work contribute, however modestly,
to improving their care and quality of life.

Acknowledgment

First and foremost, I would like to express my sincere gratitude to all those who contributed, directly or indirectly, to the completion of my academic journey over the past three years at the **Euromed University of Fez**. This period has been a formative experience both intellectually and personally.

I would like to thank the entire academic staff of **EIDIA (École d'Ingénierie Digitale et d'Intelligence Artificielle)** for their dedication to providing high-quality education and for fostering an environment that encourages learning, innovation, and personal growth.

Special thanks go to **Mrs. Loubna Ourabah**, Head of the Big Data Analytics program at EIDIA, for her continuous guidance and valuable support throughout my studies.

I am also sincerely grateful to my academic supervisor, **Ms. Sara Bakkali**, for her encouragement, insightful feedback, and constant support during the development of this final year project.

I would like to extend my gratitude to the **Sidi Mohamed Ben Abdallah University — Health Artificial Intelligence and Modeling Center (HAIM)**, Faculty of Medicine, Pharmacy and Dentistry of Fès (FMPDF), for providing an enriching research environment and access to clinical data from CHU Hassan II de Fès.

My deep appreciation goes to the nephrology team at **CHU Hassan II de Fès**, and in particular to the physicians and clinical staff who made the HD-478 cohort data available. Without their meticulous data collection spanning 2020–2024, no predictive model would have been possible.

Finally, I express my deepest appreciation to **Mr. Zakaria Alami** for his technical guidance, support, and mentorship throughout the completion of this project.

I also thank the jury members for their time and thoughtful evaluation of this work, and my fellow students and friends for the intellectual exchanges and mutual support throughout this journey.

Abstract

Keywords: Hemodialysis, Mortality Prediction, Machine Learning, Support Vector Machine, Clinical Decision Support System, Feature Engineering, LLM Preprocessing, RAG, Django, React.

Chronic kidney disease (CKD) affects over 850 million people worldwide, with a significant fraction requiring renal replacement therapy through hemodialysis. One-year mortality rates in this population remain alarmingly high (15–25%) [3, 4], driven by comorbidity burden, cardiovascular events, and nutritional decline. Clinicians currently lack integrated, data-driven tools capable of stratifying individual mortality risk at the point of care.

This report presents **AI NéphroCare**, a full-stack clinical decision support platform developed for the Nephrology Department at CHU Hassan II de Fès, Morocco. The platform integrates three main components: (1) a structured patient data management module supporting over 160 clinical fields across 11 medical sections; (2) an AI-powered data preprocessing pipeline combining a Large Language Model (LLM, Ollama qwen2.5:14b) with Retrieval-Augmented Generation (RAG, ChromaDB) for intelligent data cleaning and validation; and (3) a machine learning prediction engine trained on the HD-478 cohort (478 hemodialysis patients, 95 deaths within one year, 2020–2024).

Nine machine learning algorithms were systematically evaluated — including Logistic Regression, Support Vector Machine (SVM), Random Forest, Extra Trees, Gradient Boosting, AdaBoost, XGBoost, LightGBM, and CatBoost — using stratified 10-fold cross-validation. A **linear SVM** emerged as the best-performing model (AUC-ROC = 0.8262, PR-AUC = 0.4887, F1 = 0.5667, Brier Score = 0.1265), combining strong discriminative ability with clinical interpretability. Raw SVM probabilities are calibrated via isotonic regression (10-fold OOF) and mapped to three evidence-based clinical risk zones [3, 40, 41]:

— **Low Risk:** predicted probability $< 10\%$ (3.8% observed mortality)
— **Moderate Risk:** predicted probability between 10% and 40% (22.9% observed mortality)
— **High Risk:** predicted probability $\geq 40\%$ (49.0% observed mortality)

The platform is implemented using Django REST Framework, React 18, PostgreSQL 16, Redis, Celery, and Docker Compose, with role-based access control (RBAC) across four user profiles.

Résumé

Mots-clés : Hémodialyse, Prédiction de mortalité, Apprentissage automatique, SVM, Aide à la décision clinique, Ingénierie des features, LLM, RAG, Django, React.

La maladie rénale chronique (MRC) touche plus de 850 millions de personnes dans le monde, dont une proportion significative dépend de l'hémodialyse comme thérapie de remplacement rénal. La mortalité à un an dans cette population reste élevée (15–25%), principalement due à la charge en comorbidités, aux événements cardiovasculaires et à la dénutrition. Les cliniciens manquent d'outils intégrés et fondés sur les données pour stratifier le risque de mortalité individuel au lit du patient.

Ce rapport présente **AI NéphroCare**, une plateforme d'aide à la décision clinique développée pour le Service de Néphrologie du CHU Hassan II de Fès, Maroc. Elle intègre : (1) un module de gestion structurée des dossiers patients couvrant plus de 160 champs cliniques en 11 sections ; (2) un pipeline de prétraitement IA combinant un modèle de langage (LLM, Ollama qwen2.5 :14b) et une génération augmentée par récupération (RAG, ChromaDB) ; et (3) un moteur de prédiction ML entraîné sur la cohorte HD-478 (478 patients, 95 décès à 1 an, 2020–2024).

Neuf algorithmes d'apprentissage automatique ont été comparés. Un **SVM linéaire** s'est imposé comme le meilleur modèle (AUC-ROC = 0,8262, PR-AUC = 0,4887, F1 = 0,5667, Brier = 0,1265). Les probabilités brutes sont calibrées par régression isotonique (10-fold OOF) puis classifiées en trois zones cliniques (Faible $< 10\%$, Modérée 10–40%, Élevée $\geq 40\%$) sur la base des recommandations TRIPOD/DOPPS/KDIGO. La plateforme repose sur Django REST Framework, React 18, PostgreSQL 16 et Docker Compose.

Contents

List of Figures

List of Tables

List of Abbreviations

CDSS	Clinical Decision Support System
CKD	Chronic Kidney Disease
CRF	Chronic Renal Failure
HD	Hemodialysis
PD	Peritoneal Dialysis
CHU	Centre Hospitalier Universitaire
HAIM	Health Artificial Intelligence & Modeling Center
FMPDF	Faculté de Médecine, Pharmacie et Dentisterie de Fès
USMBA	Université Sidi Mohamed Ben Abdellah
ML	Machine Learning
SVM	Support Vector Machine
RF	Random Forest
GB	Gradient Boosting
LR	Logistic Regression
AUC	Area Under the ROC Curve
ROC	Receiver Operating Characteristic
PR	Precision-Recall
CV	Cross-Validation
LLM	Large Language Model
RAG	Retrieval-Augmented Generation
DRF	Django REST Framework
JWT	JSON Web Token
RBAC	Role-Based Access Control
API	Application Programming Interface
REST	Representational State Transfer
PTH	Parathyroid Hormone
FAV	Fistule Artério-Veineuse
KDOQI	Kidney Disease Outcomes Quality Initiative

KNN K-Nearest Neighbors

OOF Out-Of-Fold

CHAPTER 1

Overview of the Host Organization

Context, medical setting, and project objectives.

CONTENTS

Chapter 1

Overview of the Host Organization

1.1 Background

1.1.1 Digital Transformation in Healthcare Systems

The global healthcare sector is undergoing a profound digital transformation, driven by the exponential growth of medical data, the democratization of cloud computing, and the maturation of artificial intelligence (AI) technologies. Electronic Health Records (EHR) have become the standard in most developed countries, generating terabytes of structured and unstructured clinical data daily. According to the World Health Organization (WHO), the global digital health market is projected to reach $660 billion by 2025, reflecting the scale of investment in health IT worldwide [1].

In low- and middle-income countries (LMICs), including Morocco, this transition is accelerating. The Moroccan Ministry of Health's national digital health strategy (2020–2025) prioritizes the deployment of integrated hospital information systems, telemedicine platforms, and AI-based decision support tools in university hospitals. CHU Hassan II de Fès, as one of the country's largest tertiary care centers, serves as a pilot site for several of these initiatives.

Despite this momentum, a significant gap remains between data availability and clinical utility. Patient records are often fragmented across departments, stored in heterogeneous formats (paper, Excel, proprietary HIS systems), and rarely exploited for predictive analytics. Bridging this gap — from raw clinical data to actionable risk predictions — is precisely the challenge addressed by this project.

1.1.2 Importance of Data-Driven Clinical Decision Support

Clinical decision-making is inherently complex. Physicians must integrate information from dozens of variables — biological markers, comorbidities, imaging findings, treatment history — under time pressure, and with incomplete information. Cognitive

overload, variable experience, and the sheer volume of data contribute to suboptimal decisions, particularly in high-acuity settings such as nephrology and intensive care.

Clinical Decision Support System (CDSS)

A CDSS is a health information technology system designed to assist clinicians in making clinical decisions by integrating patient-specific information with a curated medical knowledge base. CDSS can be rule-based (expert systems) or data-driven (machine learning models).

Data-driven CDSS offer several advantages over traditional rule-based systems:

— **Pattern discovery**: ML models can detect non-linear, high-dimensional relationships invisible to human experts.

— **Personalization**: Predictions are tailored to the individual patient profile rather than population-average guidelines.

— **Continuous learning**: Models can be retrained as new data accumulates, adapting to evolving patient populations.

— **Explainability**: Modern interpretability tools (SHAP, feature importance) make model reasoning accessible to clinicians.

The integration of CDSS in nephrology has shown particular promise. Studies have demonstrated that early identification of high-risk dialysis patients can lead to timely interventions — nutritional support, cardiovascular optimization, transplant listing — that reduce mortality by 15–30% [5].

1.2 Host Organization

1.2.1 Sidi Mohamed Ben Abdellah University

Founded in 1975, Sidi Mohamed Ben Abdellah University (USMBA) is one of Morocco's largest public universities, headquartered in Fès. With over 100,000 students across 13 faculties and schools, USMBA covers a broad spectrum of disciplines from sciences and technology to humanities and medicine. The university has signed cooperation agreements with more than 200 international institutions and is actively engaged in research partnerships focused on health, sustainable development, and digital innovation.

1.2.2 Faculty of Medicine, Pharmacy and Dentistry (FMPDF)

The Faculty of Medicine, Pharmacy and Dentistry of Fès (FMPDF-Fès) is the academic arm responsible for training physicians, pharmacists, dentists, and health

informaticians in the region. In partnership with CHU Hassan II, the faculty provides clinical training and conducts translational research bridging laboratory science and bedside medicine. Its research units work on topics ranging from infectious diseases and oncology to health systems and biomedical informatics.

Figure 1.1 – Logo of the Faculty of Medicine, Pharmacy and Dentistry of Fès (FMPDF)

1.2.3 Health Artificial Intelligence & Modeling Center (HAIM)

The **Health Artificial Intelligence & Modeling Center (HAIM)** is a research and innovation unit operating within FMPDF under the direction of **Mr. Zakaria Alami Merrouni**. The HAIM Center focuses on the application of artificial intelligence, data science, and computational modeling to challenges in healthcare, clinical research, and medical education. Its work spans from predictive modeling in clinical settings to the development of AI-powered tools, making it a natural incubator for a project like AI NéphroCare.

HAIM's core activities include:

— Building predictive models for high-mortality conditions (chronic kidney disease, oncology, sepsis)

— Designing health data platforms that comply with clinical and ethical standards

— Training the next generation of health informaticians and clinical data scientists

— Providing decision support tools deployable in resource-limited clinical environments

As the host institution for this internship, the HAIM Center provided the problem context, the domain expertise needed to design a medically relevant platform, access to real clinical workflows, and the validation environment for testing AI NéphroCare in conditions close to production use.

Figure 1.2 – Logo of the Health Artificial Intelligence & Modeling Center (HAIM)

This project was developed within HAIM, in direct collaboration with the Nephrology

Department of CHU Hassan II de Fès, ensuring clinical relevance and real-world applicability.

1.3 Medical Context

1.3.1 Kidney Disease and Dialysis

Anatomy and Function of the Kidney

The kidneys are paired retroperitoneal organs responsible for maintaining homeostasis through filtration of approximately 180 liters of blood per day [39]. Their principal functions include:

— Excretion of metabolic waste products (urea, creatinine, uric acid)

— Regulation of fluid and electrolyte balance (sodium, potassium, calcium, phosphate)

— Acid-base homeostasis (bicarbonate reabsorption)

— Endocrine functions: renin secretion (blood pressure), erythropoietin (red blood cell production), vitamin D activation [3]

Chronic Kidney Disease (CKD)

CKD is defined as structural or functional kidney abnormalities persisting for more than three months, with implications for health. The **KDIGO 2012 classification** stages CKD into five categories based on glomerular filtration rate (GFR):

Table 1.1 – CKD staging by KDIGO 2012

Stage	Description	GFR (mL/min/1.73m^2)	Prevalence
G1	Normal/high	≥ 90	∼3.5%
G2	Mildly decreased	60–89	∼3.9%
G3a	Mildly to moderately decreased	45–59	∼7.6%
G3b	Moderately to severely decreased	30–44	∼3.4%
G4	Severely decreased	15–29	∼0.4%
G5	Kidney failure	< 15	∼0.1%

At stage G5 (end-stage renal disease, ESRD), renal replacement therapy (RRT) becomes necessary. The three modalities of RRT are: hemodialysis (HD), peritoneal dialysis (PD), and kidney transplantation. Globally, over 2.6 million people are on RRT, with HD being the most prevalent modality (∼70% of RRT patients).

Hemodialysis: Principle and Clinical Course

Hemodialysis works by passing the patient's blood through an artificial membrane (dialyzer) which removes waste products and excess water via diffusion and ultrafiltration. A typical HD session lasts 4 hours, 3 times per week.

The quality of HD is measured primarily by the **Kt/V** index, where K is the dialyzer clearance, t the session duration, and V the urea distribution volume. A Kt/V ≥ 1.2 per session is considered adequate per KDIGO guidelines.

Epidemiology in Morocco

In Morocco, the national registry (MAGREDIAL) recorded approximately 16,000 patients on chronic hemodialysis in 2022, with an annual incidence of 3,500 new patients. The Fès-Meknès region, served by CHU Hassan II, accounts for ∼18% of national HD patients. The dominant etiologies of CKD in Morocco are:

1. Diabetic nephropathy (32%)
2. Hypertensive nephrosclerosis (28%)
3. Undetermined (18%)
4. Glomerulonephritis (12%)
5. Other (10%)

1.3.2 Mortality Risk Challenges

Despite advances in dialysis technology, the annual mortality rate of HD patients remains strikingly high — approximately **15–25%** in most national registries [3, 4, 41]. By comparison, this exceeds the 5-year mortality of many solid tumors.

The principal causes of death in HD patients are:

— **Cardiovascular disease** (40–50%): sudden cardiac death, acute coronary syndrome, heart failure

— **Infections** (15–20%): vascular access-related sepsis, pneumonia

— **Stroke** (5–10%)

— **Withdrawal from dialysis** (10–15%, predominantly in elderly patients)

Key mortality predictors identified in the literature include: low serum albumin (< 35 g/L), high Charlson Comorbidity Index, high PTH, tunneled catheter use (vs. arteriovenous fistula), reduced residual diuresis, and low dialysis frequency [3, 4].

1.4 Problem Statement

1.4.1 Raw Hospital Data is Underutilized

At CHU Hassan II, the nephrology unit has accumulated several years of longitudinal patient records — biological results, dialysis session data, complication histories — primarily in paper records and Excel spreadsheets. This data is rich, but entirely siloed: it cannot be queried, aggregated, or used for risk prediction without significant manual effort. The absence of a unified digital platform means that potentially life-saving information lies dormant.

1.4.2 Lack of Predictive Tools for Mortality Risk

Clinicians currently rely on subjective assessment and population-level guidelines to estimate individual mortality risk. Validated scores such as the **Charlson Comorbidity Index** or the **Davies score** provide only coarse stratification and do not capture the full multivariate complexity of HD patient profiles. No locally validated, ML-based risk prediction tool was available to the CHU Hassan II nephrology team prior to this project.

1.4.3 Data Heterogeneity and Quality Issues

The HD-478 cohort dataset was collected over 4 years by multiple clinicians, resulting in considerable heterogeneity: inconsistent decimal separators (comma vs. period), variable biological units (mg/dL vs. mmol/L), free-text fields where categorical values are expected, and missing values in up to 30% of certain variables. Before any ML model could be trained, a robust data preprocessing pipeline was essential.

1.5 Project Objectives

1.5.1 General Objective

Design, implement, and validate a full-stack intelligent platform that enables the Nephrology Department of CHU Hassan II to (1) centralize and manage hemodialysis patient data, (2) automatically preprocess and validate imported datasets using AI, and (3) predict 1-year mortality risk for individual patients using a validated machine learning model integrated into a clinical decision support interface.

1.5.2 Specific Objectives

1. Build a structured patient data model capturing 160+ clinical variables across 11 medical sections (demographics, biology, dialysis quality, comorbidities, complications, outcomes).
2. Develop an LLM+RAG preprocessing pipeline that automatically detects and corrects data quality issues in imported Excel files.
3. Train and compare **nine** machine learning algorithms on the HD-478 cohort, selecting the best-performing model for production deployment.
4. Engineer 32 clinically meaningful features (24 raw clinical + 8 interaction features) from the raw dataset.
5. Select the optimal classification threshold by maximizing F1-score, and define three clinically actionable risk zones (Low/Moderate/High) with fixed boundaries validated by the nephrology team.
6. Implement a secure, multi-role web application with full audit trail, role-based access control, and clinical visualization dashboards.
7. Deploy the entire system as a reproducible Docker Compose stack.

1.6 Methodology

1.6.1 Agile Approach

The project followed an **Agile Scrum** methodology adapted to an academic PFE context. The development was organized into bi-weekly sprints, each delivering a testable increment of the platform. The Scrum workflow comprised:

— **Sprint Planning**: Definition of user stories and acceptance criteria
— **Daily Standups**: Brief progress check with supervisor
— **Sprint Review**: Demonstration of completed features to the nephrology team
— **Sprint Retrospective**: Identification of process improvements

1.6.2 Iterative Development

Five main phases were completed iteratively, starting from an essential preparatory phase before any development began:

1. **Phase 0 — Documentation & Skill Building** (Month 1):
   — *Bibliographic research*: systematic review of scientific articles on hemodialysis mortality prediction, clinical decision support systems, and ML in nephrology
   — *Medical domain acquisition*: understanding of nephrology concepts (CKD staging, dialysis modalities, mortality risk factors, Charlson index, Kt/V)

— *Technical upskilling*: completion of Data Science training covering pandas, scikit-learn, Django REST Framework, and React

— *Alignment*: mapping medical domain knowledge onto project technical objectives

2. **Phase 1** (Months 1–2): Data modeling, database design, backend skeleton, authentication
3. **Phase 2** (Months 2–3): Patient CRUD, import/export, LLM preprocessing pipeline
4. **Phase 3** (Months 3–4): ML model training, comparison, feature engineering, calibration
5. **Phase 4** (Months 4–5): Frontend interfaces, prediction UI, deployment, testing

1.6.3 Task Management with Trello

Tasks were tracked using a **Trello** board, a visual project management tool based on the Kanban methodology. The board was structured to reflect the full lifecycle of each task, from initial idea to validated delivery. It included the following columns:

— **Backlog**: all identified tasks and user stories pending prioritization

— **In Progress**: tasks currently under active development

— **Review**: completed tasks awaiting code review or supervisor feedback

— **Done**: fully validated and merged tasks

— **Meetings**: agenda items and decisions recorded from weekly meetings with supervisors

— **To Validate (Technical Supervisor)**: tasks requiring technical validation by Mr. Zakaria Alami before integration

— **Validated**: tasks approved after supervisor review

— **Project Tracking**: milestones, deadlines, and sprint-level progress summaries

Each card included a technical description, acceptance criteria, priority level, and estimated effort in story points. This multi-column organization provided full transparency to both the academic and technical supervisors, and facilitated continuous prioritization of clinical and technical requirements throughout the project.

Figure 1.3 – Trello board used for task management and project tracking

CHAPTER 2

State of the Art

Review of existing CDSS, ML models, and data engineering approaches.

CONTENTS

Chapter 2

State of the Art

2.1 Clinical Decision Support Systems

Clinical Decision Support Systems (CDSS) have evolved from simple rule-based alert systems to sophisticated AI-driven platforms [23]. The earliest CDSS — such as MYCIN (1970s) [24] and INTERNIST-1 [25] — relied on manually coded expert rules. Modern CDSS leverage machine learning to discover patterns in large clinical datasets [23].

A landmark meta-analysis by Bright et al. (2012) reviewed 148 RCTs of CDSS and found that computerized clinical decision support improved clinical process measures in 68% of trials and patient outcome measures in 57% [2]. However, challenges persist: alert fatigue, poor EHR integration, and lack of local validation limit real-world adoption [2].

In nephrology specifically, CDSS have been applied to: (1) drug dosing adjustment for renal function, (2) dialysis adequacy monitoring, (3) anemia management [3], and (4) mortality risk stratification [5]. The latter remains the most challenging due to the complexity of HD patient comorbidity profiles.

2.2 Healthcare Data Engineering

The extraction, transformation, and loading (ETL) of healthcare data presents unique challenges:

— **Structural heterogeneity**: Data may be structured (lab values, vital signs), semi-structured (clinical notes), or unstructured (radiology reports).

— **Missing values**: Clinical datasets routinely exhibit 20–50% missingness, driven by test ordering variability and documentation practices.

— **Temporal dynamics**: Patient state evolves over time; longitudinal aggregation is non-trivial.

— **Coding variability**: The same concept may be encoded differently across sites (ICD-10, SNOMED-CT, free text).

Recent advances in **Large Language Models (LLMs)** have opened new possibilities for clinical data cleaning. Models such as GPT-4, LLaMA, and Qwen can parse free-text clinical records, standardize terminologies, and detect anomalous values — tasks previously requiring specialized NLP pipelines [7]. **Retrieval-Augmented Generation (RAG)** enhances LLMs by grounding their responses in a domain-specific knowledge base, reducing hallucinations and improving precision on structured extraction tasks [8].

2.3 Machine Learning in Mortality Prediction

A substantial body of literature has investigated ML approaches to mortality prediction in CKD and HD populations:

Table 2.1 – Selected ML studies on hemodialysis mortality prediction

Study	Cohort	Model	AUC	Outcome
Couchoud et al. [26]	REIN (France)	LR	0.72	1-year mortality
Wick et al. [27]	USRDS	RF + GBM	0.81	1-year mortality
Kaesler et al. [28]	German HD	LSTM	0.78	90-day mortality
Zhu et al. [29]	Chinese cohort	XGBoost	0.83	1-year mortality
Nasr et al. [30]	Egyptian HD	SVM	0.79	6-month mortality
This work	HD-478 (Fès)	SVM Linear	**0.826**	1-year mortality

Several trends emerge from this literature: (1) ensemble methods (RF, GBM, XGBoost) generally outperform linear models on large datasets but may overfit on small cohorts; (2) linear models like SVM and LR offer superior interpretability and are preferred in clinical settings; (3) the importance of calibration is increasingly recognized — a model with high AUC but poorly calibrated probabilities is of limited clinical use.

2.4 Feature Engineering in Medical Data

Feature engineering — the process of creating informative input representations from raw clinical data — is often the single most impactful step in a medical ML pipeline. In the HD mortality prediction literature, the most consistently important features are:

— **Serum albumin**: A low albumin (< 35 g/L) is the most robust single predictor of HD mortality across all published studies, reflecting the synergistic effects of malnutrition and chronic inflammation [6].

— **Charlson Comorbidity Index (CCI)**: Validated in HD populations [14]; each unit increase is associated with a 10–15% increased mortality risk [31].

— **Vascular access type**: Arteriovenous fistula (AVF) use is a strong protective factor; tunneled catheters are associated with 2–3× higher infection and mortality risk [32].

— **Residual renal function**: Even small amounts of residual diuresis (>100 mL/24h) are associated with significantly lower cardiovascular mortality [33].

— **Interaction features**: Non-linear combinations (e.g., age × CCI, albumin × hospitalizations) capture patient fragility beyond what individual variables convey [27].

2.5 Existing Dialysis Prediction Models

Several commercial and academic tools exist for HD risk prediction:

— **USRDS Predictive Model** (USA) [34]: Logistic regression on registry data; AUC ≈ 0.72; not validated outside the US.

— **REIN Score** (France) [4]: Risk score derived from the French ESRD registry; validated in European populations; requires 8 input variables.

— **Khan Score** [35]: CCI-derived; simple but underperforms with AUC ≈ 0.68 in most external validations.

— **DIAL-risk** (Spain) [36]: Random Forest on Spanish HD registry; AUC = 0.80; proprietary, not deployable locally.

None of these tools are validated in Moroccan or North African HD populations, nor do they integrate with a clinical data management platform.

Although several Moroccan studies have explored the clinical characteristics and mortality risk factors of hemodialysis patients [37, 38], the majority of these works remain descriptive and retrospective. To date, few studies have applied advanced machine learning techniques for individualized mortality risk prediction, and no validated clinical decision support system has been widely deployed in Moroccan hospital centers. This gap highlights the need for a locally trained, end-to-end platform such as AI NéphroCare.

2.6 Critical Analysis

2.6.1 Limitations of Existing Systems

1. **External validity**: Most published models are trained on large Western registries (USRDS, REIN) with demographic and etiological profiles significantly different

from Moroccan patients (higher proportion of young diabetic patients, lower transplantation rates, different socioeconomic determinants).

2. **Deployment gap**: Published models rarely translate into operational clinical tools. The gap between a trained model and a deployed clinical application involves authentication, data management, UI design, and regulatory compliance — aspects largely absent from academic ML papers.
3. **Data quality**: Most studies assume clean, complete datasets. In practice, clinical data requires extensive preprocessing before ML models can be applied.
4. **Interpretability**: Black-box models (deep neural networks) may achieve high AUC but cannot explain their predictions to clinicians, limiting trust and adoption.

2.6.2 Positioning of the Proposed Solution

AI NéphroCare addresses these limitations by:

— Training on a **locally collected cohort** from the same clinical environment where the tool will be deployed

— Providing a **complete clinical platform** — not just a model — integrating data management, preprocessing, prediction, and audit

— Selecting a **linear SVM** for its interpretability and robustness on imbalanced datasets with high-dimensional feature spaces

— Implementing **probability calibration** to ensure predicted scores correspond to actual observed mortality rates

— Providing **clinical reasoning** alongside each prediction (top-5 contributing factors, risk zone, tailored recommendation)

CHAPTER 3

Global System Architecture

Platform overview, layered design, and technology stack.

CONTENTS

Chapter 3

Global System Architecture

3.1 Overview of the Platform

AI NéphroCare is a full-stack web application organized around three core modules:

1. **Patient Data Management**: Structured storage, CRUD operations, import/export of clinical data for hemodialysis patients.
2. **AI Preprocessing Pipeline**: LLM-assisted data cleaning with RAG-based context retrieval, cell-level corrections, and supervised validation.
3. **ML Prediction Engine**: 32-feature SVM mortality model with calibrated probabilities and three-zone risk stratification.

These modules are exposed through a Django REST API consumed by a React single-page application, with all services containerized via Docker Compose.

Use Case Diagram

Figure 3.1 presents the use case diagram of the platform. Four actors interact with the system according to their role. All authenticated users can import Excel files and trigger the LLM preprocessing pipeline; however, the final validation and integration of data into the platform is restricted to Super Admin and Chef de Service.

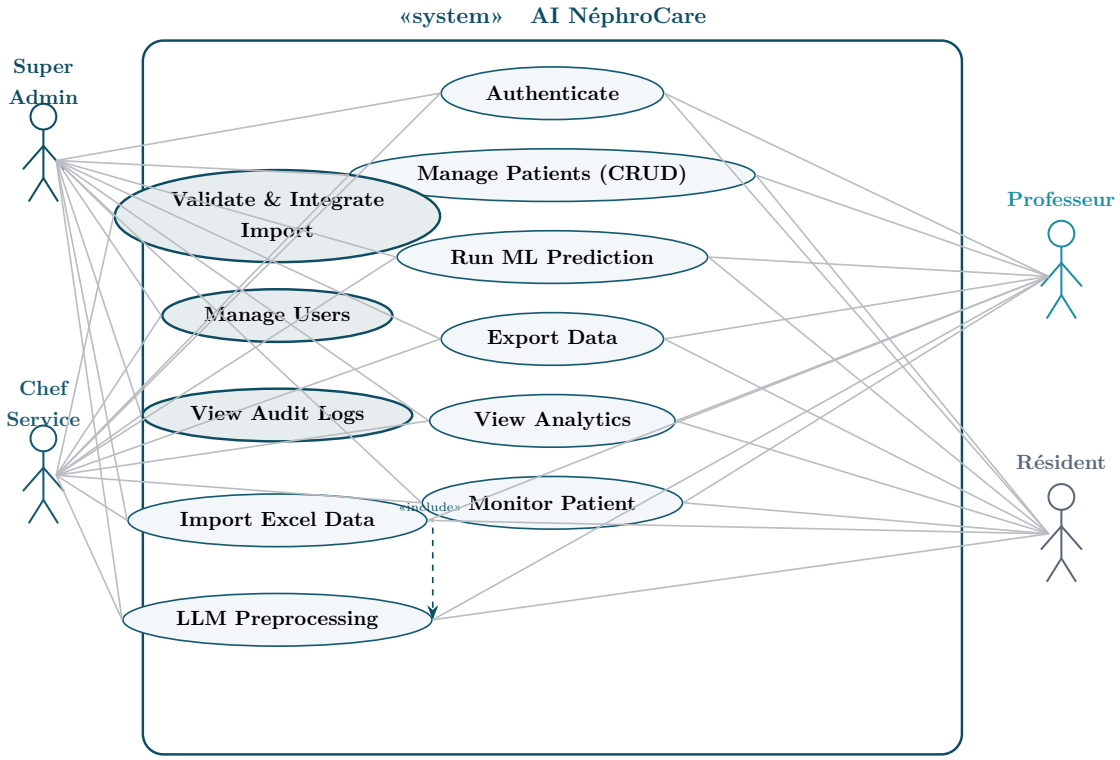

Figure 3.1 – Use Case Diagram — AI NéphroCare (4 actors, 11 use cases). Professeur and Résident have identical access. Validate & Integrate Import is restricted to Super Admin and Chef de Service.

3.2 Architectural Style

3.2.1 Full-Stack Architecture

The platform follows a classical three-tier architecture:

— **Presentation tier**: React 18 SPA running in the browser (served via Node/serve on port 3000)

— **Application tier**: Django REST Framework API (port 8000) containing all business logic, ML inference, and preprocessing

— **Data tier**: PostgreSQL 16 (port 5432) for persistent storage; Redis 7 (port 6379) for Celery task queue and session cache

3.2.2 Integrated Data Pipeline

The data pipeline is fully embedded within the Django backend. It handles the complete data lifecycle in three sequential stages:

1. **Ingestion**: Excel/CSV files uploaded by users are parsed by pandas into a structured DataFrame. Column names are normalized and a session identifier is assigned.

2. **AI Preprocessing**: The DataFrame is passed asynchronously (via Celery) to the LLM preprocessing engine, which combines Ollama qwen2.5:14b (GPU) with a ChromaDB RAG knowledge base to detect and correct clinical data quality issues at the cell level.
3. **Integration**: After review and validation by an authorized user (Super Admin or Chef de Service), the corrected data is mapped to the patient data model and persisted in PostgreSQL.

The complete technical implementation of this pipeline is detailed in Chapter 4.

3.3 Layered Architecture

AI NéphroCare follows a five-layer architecture where each layer has a clear responsibility. Figure 3.2 presents this architecture with the key technologies at each level. Solid arrows represent the main synchronous data flow (top-down request/response). The two dashed pink arrows represent secondary, asynchronous data paths: the left one shows that the API layer (L2) directly triggers the Data Engineering layer (L4) for long-running preprocessing tasks via Celery, bypassing the ML layer; the right one shows that the ML layer (L3) reads serialized model files directly from the Data layer (L5) at inference time.

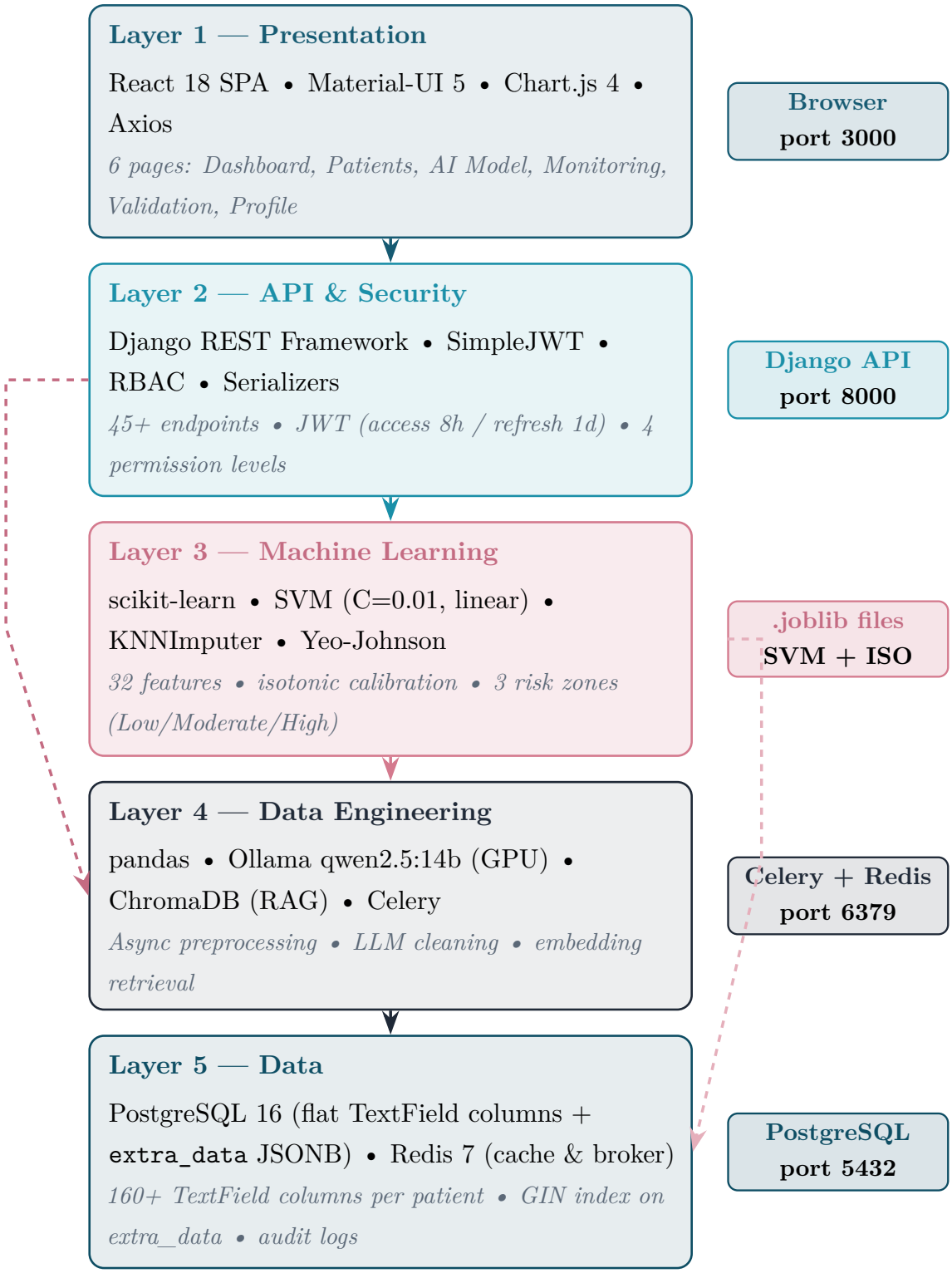

Figure 3.2 – Five-layer architecture of AI NéphroCare with technologies per layer

Table 3.1 details the specific responsibility of each layer and its communication interfaces. L1 communicates exclusively with L2 via HTTP REST calls; L2 orchestrates both L3 (for synchronous ML predictions) and L4 (for asynchronous preprocessing jobs); L3 and L4 both persist and retrieve data from L5, which serves as the single source of truth for all clinical records and model artifacts.

Table 3.1 – Layer responsibilities and inter-layer communication

Layer	Name	Responsibility	Communicates with
L1	Presentation	Clinical UI, charts, forms	L2 via REST/JSON (Axios)
L2	API & Security	Routing, auth, serialization	L1 (HTTP), L3, L4 (Python)
L3	Machine Learning	Feature extraction, SVM, calibration	L2 (results), L5 (models)
L4	Data Engineering	LLM preprocessing, async tasks	L2 (triggers), L5 (storage)
L5	Data	Persistent storage, caching	L3, L4 (SQL/Redis)

3.4 Technology Stack

Table 3.2 – Complete technology stack

Category	Technology	Version	Role
Backend	Python	3.11	Primary language
	Django	6.0	Web framework, ORM
	Django REST Framework	3.15	REST API, serialization
	SimpleJWT	5.3	JWT authentication
	scikit-learn	1.4	SVM, imputation, calibration
	pandas / numpy	2.2 / 1.26	Data manipulation
	Celery	5.3	Async task queue
	ChromaDB	latest	Vector database (RAG)
Frontend	React	18	UI framework
	Material-UI	5.15	Component library
	Chart.js	4	Clinical charts
	axios	latest	HTTP client
	react-router-dom	6	Client-side routing
Infrastructure	Docker / Compose	24+ / 2+	Containerization
	PostgreSQL	16	Relational database
	Redis	7	Cache & broker
	Ollama	latest	Local LLM server
	Nginx/serve	latest	Static file serving

3.4.1 Backend: Django REST Framework

Django was chosen for its mature ORM, built-in admin interface, and the DRF extension which dramatically accelerates REST API development. The `JSONB` support in Django's PostgreSQL backend allows flexible storage of heterogeneous clinical data without sacrificing query capability.

3.4.2 Frontend: React

React's component-based architecture maps naturally to the modular clinical interface requirements. The Context API provides lightweight state management for authentication and language preferences without the overhead of Redux.

3.4.3 Database: PostgreSQL

PostgreSQL's `JSONB` type — which stores JSON data in a binary format and supports indexing — was critical for storing the 11 clinical sections of each patient record without a rigid schema that would break with each new clinical variable. GIN indexes on JSONB columns maintain query performance.

3.4.4 Containerization: Docker

All seven services (frontend, backend, database, Redis, Ollama, Celery worker, audit cleanup) are defined in a single `docker-compose.yml` file, enabling one-command deployment on any Docker-capable host. This is essential for a hospital environment where IT staff may not have deep DevOps expertise.

CHAPTER 4

Data Engineering Pipeline

From raw clinical records to ML-ready features: curation, storage, and preprocessing.

CONTENTS

Chapter 4

Data Engineering Pipeline

4.1 Data Sources

4.1.1 Hospital Patient Records

The primary data source is the **HD-478 cohort**: a retrospective dataset of 478 hemodialysis patients treated at CHU Hassan II de Fès between 2020 and 2024. The cohort was assembled by nephrologists from paper records, biological reports, and the hospital information system.

Key characteristics:

— 478 patients (383 alive, 95 deceased at 1 year — 19.9% mortality)

— Follow-up period: from dialysis initiation to 1-year outcome or censoring

— Variables collected: 160+ across demographics, biology, dialysis modality, comorbidities, complications, and outcome

4.1.2 Excel / CSV Imports

The platform is designed to accept Excel (`.xlsx`) and CSV files exported from hospital systems. Column names and formats vary between sources, necessitating a flexible ingestion pipeline.

4.1.3 Pre-Computed Clinical Attributes

The HD-478 dataset includes several clinical variables that were recorded directly in the patient's medical dossier (*dossier médical*) by the clinical team and retrieved as-is — they were not computed by the platform:

— **Age**: The patient's age at dialysis initiation, as documented in the clinical record.

— **Kt/V**: A dialysis adequacy index recorded in the dialysis session reports.

— **Charlson Comorbidity Index (CCI)** [14]: A standardized comorbidity score documented in the patient file. The CCI assigns weighted scores (weights $w_i \in \{1, 2, 3, 6\}$) to 17 comorbidity categories present in the patient record, making it a rich summary of disease burden retrieved directly from the clinical dossier.

4.2 Data Ingestion

4.2.1 File Upload Endpoints

Files are uploaded via `POST /api/patients/import/` (multipart/form-data). The backend:

1. Validates the file extension and MIME type (Multipurpose Internet Mail Extensions)
2. Reads the file into memory using pandas and openpyxl
3. Computes a column profile (names, types, null rates)
4. Creates a preprocessing session (`session_id`) and returns it to the frontend

4.2.2 Parsing with Pandas

This subsection describes how the platform uses the pandas library to read the uploaded file into a DataFrame, normalizing column names by stripping whitespace and converting to lowercase. This makes the data structure uniform regardless of source file formatting.

Once parsed, the platform immediately computes a **technical profile** of the uploaded DataFrame — a structured summary of column statistics (names, types, missing rates, unique counts, sample values) used to drive the LLM routing decision (Section 4.6).

4.2.3 Import-Time Normalization

After the file is parsed, the platform applies lightweight normalization before any further processing:

— **Decimal separators**: European commas (e.g., `3,5`) are replaced by decimal points

— **Sex encoding**: Values `M`, `m`, `Masculin`, `H`, `1` → unified encoding; similarly for female

— **Boolean fields**: Yes/No, Oui/Non, 0/1 mapped to a consistent binary format

— **Structural exclusions**: Columns with $> 90\%$ missingness (3 identified: `transplantation_renale` `distance`, `acces`) are flagged and excluded from ML features

— **Date parsing**: Multi-format parser tries standard date formats first, then falls back to Excel serial number conversion (days since 1899-12-30)

4.3 Manual Data Curation of the HD-478 Dataset

Before any machine learning preprocessing could be applied, the raw dataset underwent an extensive manual curation phase. The resulting corrected dataset serves as the sole input to the ML pipeline. This section documents all nine categories of corrections applied.

4.3.1 Overview of Modifications

A line-by-line and column-by-column comparison of the two files identified exactly **9 categories of corrections**. Table 4.1 summarizes the global state of the dataset before and after curation.

Table 4.1 – HD-478 dataset: original vs. corrected dataset

Indicator	**Original**	**Corrected**
Number of patients	478	478
Number of columns	91	91
Data types	Mixed (text, int, serial dates)	Normalized (float64, datetime64)
NaN → 0 (transplant)	∼300 NaN per column	Imputed logically (0)
DFGe MDRD	105 zeros + outliers (9, 11, 12)	Recalculated (109 values)
Biological outliers	11 cells	Corrected
X/x markers	>275 cells (10 columns)	Replaced by NaN
Text → float	>130 cells	Converted to float64
Serial Excel dates	480 integer dates	Converted to datetime64

4.3.2 Column Restructuring

Five columns were relocated within the corrected dataset to group derived and administrative features together: `couverture_medicale`, `annee_inclusion`, `diabete`, `dyspnee`, `oedemes_surcharge`. This grouping ensures a logical separation between clinical measurement variables and contextual patient attributes, which facilitates downstream feature selection.

4.3.3 Date Conversion from Excel Serial Numbers

Both date columns (`date_debut_dialyse` and `date_deces`) were stored as Excel serial integers (days since 1900-01-01), making them unreadable and unusable for survival analysis. All **480 dates** were converted to `datetime64` objects.

Table 4.2 – Date conversion examples

Variable	Original	Corrected	N corrected
`date_debut_dialyse`	45261 (integer)	2023-12-01	478 (all)
`date_deces`	44214 (integer)	2021-01-18	2

Note

Critical error detected — Line 31: `date_debut_dialyse` contained the Excel serial `410381`, corresponding to year **3023-08-01** (typo: 3023 instead of 2023). Corrected to 2023-08-01.

4.3.4 Correction of Biological Outliers

Eleven cells exhibited biologically impossible values due to unit errors or data entry mistakes. The following clinical reference ranges were used to identify and correct these outliers [45]:

- Bicarbonates: 22–29 mmol/L
- Calcium: 80–105 mg/dL
- Phosphate: 30–120 mg/dL
- Creatinine (dialysis patient): >200 mg/L

Table 4.3 – Biological outliers corrected in the corrected dataset

Variable	Excel row	Original	Corrected	Error type
`bicarbonates_basaux`	160	50.0 mmol/L	15.0	Unit error (×3)
`bicarbonates_basaux`	219	42.0	14.0	Unit error (×3)
`bicarbonates_basaux`	248	56.0	16.0	Unit error (×3)
`calcium_basale`	91	5.8 mg/dL	58.0	Missing ×10 factor
`calcium_basale`	99	8.0 mg/dL	80.0	Missing ×10 factor
`phosphore_basale`	22	10.0 mg/L	100.0	Missing ×10 factor
`phosphore_basale`	39	12.0	120.0	Missing ×10 factor
`phosphore_basale`	48	10.0	100.0	Missing ×10 factor
`phosphore_basale`	296	8.0	80.0	Missing ×10 factor
`phosphore_basale`	335	252.0	52.0	Outlier (252 → 52)
`creatinine_basale`	335	30 mg/L	300	Missing digit (30 → 300)

4.3.5 DFGe MDRD Recalculation (109 values)

The `dfge_mdrd` column contained 105 values coded as 0 (impossible for a living patient) and several biological outliers (values of 9, 11, 12). All 109 erroneous values were recalculated using the **simplified MDRD-4 formula** (IDMS-standardized, factor 175), recommended by nephrology learned societies:

$$\text{DFGe [mL/min/1.73m}^2\text{]} = 175 \times S_{cr}^{-1.154} \times \text{Age}^{-0.203} \times \begin{cases} 0.742 & \text{if female} \\ 1.000 & \text{if male} \end{cases} \tag{4.1}$$

where S_{cr} is serum creatinine in mg/dL (converted from mg/L by dividing by 10), and Age is in years. The constant 175 replaces the older 186 in the IDMS-standardized version [12]. The ethnic correction factor (1.212 for African-American patients) was not applied as this information was unavailable in the dataset.

Table 4.4 shows a representative sample of the 109 corrections:

Table 4.4 – Sample DFGe MDRD corrections (109 total)

Excel row	Original DFGe	Corrected DFGe	Status
4	0	5.0	Impossible value (0) → recalculated
7	0	2.8	Impossible value (0) → recalculated
15	0	16.1	Impossible value (0) → recalculated
342	12	5.8	Incoherent value → recalculated
356	9	3.4	Incoherent value → recalculated
369	11	2.8	Incoherent value → recalculated
… + 103 other rows (total 109)			

4.3.6 Text-to-Numeric Conversions

Several columns stored numeric values as text strings or contained heterogeneous formats (text numbers, x markers, datetime objects). Each subcategory below documents the corrections applied column by column.

Proteinuria (`proteinurie_basale`) — 127 corrections

This column stored numeric values as character strings (e.g., `'5.17'` instead of 5.17), as well as `'x'`/`'X'` markers and one mistyped integer (412 → 4.12 after dividing by 100). All 127 values were converted to `float64` or `NaN`. Correct numeric encoding is essential for this variable as proteinuria level is a clinically significant predictor of CKD progression.

Vitamin D (`vitamine_d_basale`) — 6 corrections

Mix of text values, marker `'x'`, and invalid zero: rows 98, 128, 206, 392 (text → float64); row 261 (`'x'` → `NaN`); row 316 (0 → `NaN`). Vitamin D deficiency is prevalent in dialysis patients, making its numeric encoding critical for downstream ML feature computation.

Pre-dialysis follow-up (`suivi_predialyse_mois`) — 5 corrections

Heterogeneous formats: row 9 (0 → 72, recovered from records); row 278 (`'1mois'` → 1.0); rows 327 and 421 (`datetime.time(0,0)` → 0.0); row 420 (erroneous Excel 1900 date → 72.0). The duration of pre-dialysis nephrology follow-up is an important contextual variable that must be numeric for survival analysis.

4.3.7 Cleaning of 'X'/'x' Markers in Education Columns

In therapeutic education columns, cells were filled with `'X'` or `'x'` instead of binary 0/1 values. Since the exact meaning (presence = 1? non-applicable?) could not be determined without patient record review, all **265 markers** were replaced by `NaN`.

Table 4.5 – X/x marker cleanup by column (265 total)

Column	N markers	Action
`comprehension_maladie`	39	→ NaN
`connaissance_therapie_substitution`	40	→ NaN
`connaissance_pratique_dialyse`	40	→ NaN
`education_soins_acces`	40	→ NaN
`surveillance_hydrique_ponderale`	40	→ NaN
`education_regime`	40	→ NaN
`education_traitements_associes`	11	→ NaN
`education_complications`	12	→ NaN
`ferritine_basale`	2	→ NaN
`seances_par_semaine`	1	→ NaN
Total	**265**	All → NaN

4.3.8 Logical NaN Imputation for Transplant Binary Variables

For four transplant-related binary variables, missing values (NaN) were interpreted clinically as *"not performed"*: absence of documentation equals absence of the clinical act. Consequently, **1,261 NaN values** were replaced by 0.

In the clinical context of hemodialysis patient records, the absence of documentation for transplant-related procedures is itself clinically informative. When no pre-transplantation workup, blood transfusion, transplant information session, or waiting list entry was recorded, this absence reflects the clinical reality that these acts were never initiated — not that the data was simply not collected. This zero-imputation is therefore not a statistical guess but a clinically grounded decision: *non-documentation equals non-event* in this administrative context. This approach is consistent with standard practices in retrospective clinical data curation [3].

Table 4.6 – NaN → 0 imputation for transplant variables

Variable	N imputed	Clinical justification
`bilan_pretransplantation`	349	No documentation = workup not performed
`immunisation_transfusion_sanguin`	307	No documentation = no transfusion on record
`information_transplantation_donneu`	304	No documentation = information not provided
`liste_attente_transplantation`	301	No documentation = not on waiting list
Total	**1,261**	

Special case: Row 63 in `liste_attente_transplantation` contained the string `'1;(1/2024)'` — cleaned to 1 (patient registered in January 2024).

4.3.9 Logical Recodings and Modality Corrections

Beyond numeric corrections, the dataset contained 14 categories of logical inconsistencies — cases where documented values contradicted other clinical information in the same record. Each recoding category below describes the nature of the error, how it was identified, and the correction applied.

— **15 asymptomatic patients** recoded from 1 to 0: patients coded as asymptomatic but presenting documented symptoms

— **Hemodialysis variable** (4 rows): incorrectly coded 0 → 1

— **Vascular access corrections**: 5 rows with `cathetere_femoral` NaN → 1 (confirmed in records); 2 rows 1 → 0

— **Binary recoding**: values > 1 (infection=2, hemorrhage=2, access dysfunction=3) → 1

— **Cause of death**: row 396, `'OUI'` → 1.0

Each of these corrections was verified against the original paper-based clinical records to ensure that no information was introduced or fabricated — all changes are conservative recodes toward clinically consistent representations.

4.3.10 Final Dataset Structure

After all corrections, the corrected dataset presents the following variable type distribution:

Table 4.7 – Variable type distribution in the corrected dataset (ML-ready)

Category	N variables	%	Examples
Binary variables (0/1)	57	62.6%	sex, death, hypertension, HD, infection
Continuous (float/int)	21	23.1%	age, Charlson, DFGe, albumin, creatinine
Ordinal categorical	11	12.1%	access type, death cause, coverage
Date (datetime64)	2	2.2%	dialysis start, death date
100% empty (exclude)	3	3.3%	transplantation_renale, distance, acces
Total	**91**	100%	

4.3.11 Quantitative Summary of Manual Curation

Manual Curation — Complete Bilan

- 9 categories of corrections applied
- **480 dates** converted (Excel serial → datetime64)
- **109 DFGe values** recalculated via MDRD formula
- **1,261 NaN → 0** on 4 binary transplant variables
- **127 proteinuria values** converted (text → float + x cleanup)
- **265 X/x markers → NaN** across 10 columns
- **11 biological outliers** corrected (unit / entry error)
- **15 asymptomatic patients** re-coded to 0
- **14 vascular access / dialysis modality** corrections
- 3 100%-empty columns identified for ML exclusion

4.4 Data Storage & Schema Design

4.4.1 Entity-Relationship Diagram

The platform's persistence layer is designed around a PostgreSQL 16 relational database. Figure 4.1 presents the Entity-Relationship diagram showing the five main

entities and their associations. Each entity box lists its key attributes; underlined attributes are primary keys; italicised attributes are foreign keys.

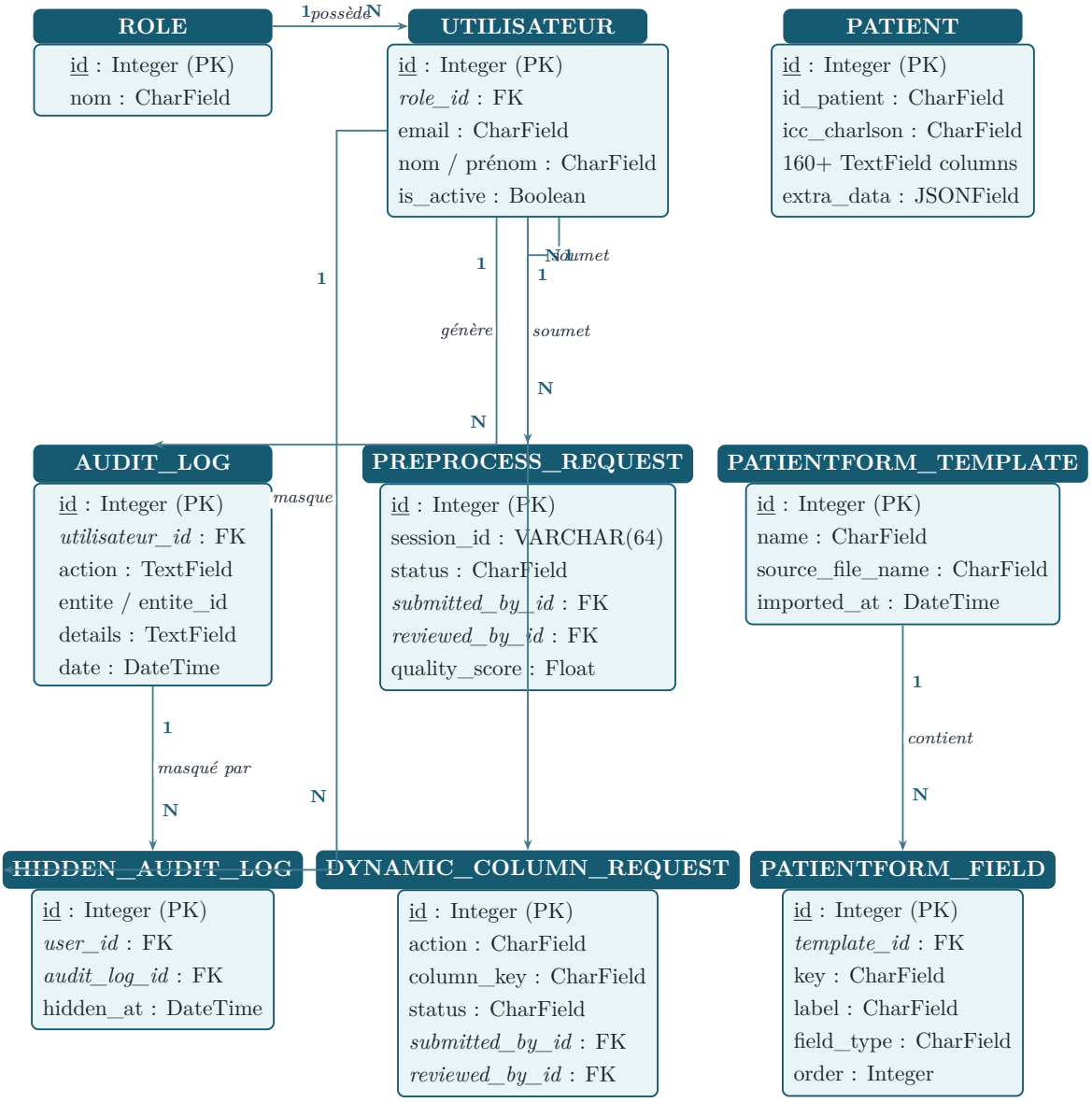

Figure 4.1 – Entity-Relationship diagram — AI NéphroCare complete database schema (PostgreSQL 16). 9 custom tables. Underlined = primary key; italicised = foreign key.

4.4.2 PostgreSQL Schema

The database `plateforme_medicale` (PostgreSQL 16) contains nine custom tables created by Django's migration system. These tables are divided into two categories:

— **Core tables** (4): the tables that form the fundamental data model of the platform — users, roles, patients, and audit trail. All clinical data, access control, and traceability pass through these tables.

— **Auxiliary tables** (5): tables that support specific platform features (LLM preprocessing workflow, dynamic column management, form templates, hidden

audit entries). They extend the core model without altering it.

Table 4.8 provides a complete overview of all nine tables.

Table 4.8 – All tables in the `plateforme_medicale` database — core vs. auxiliary

Table (PostgreSQL)	Type	Group	Role
`users_role`	**Core**	Authentication	Available user roles
`users_utilisateur`	**Core**	Authentication	User accounts, hashed password, RBAC
`patients_patient`	**Core**	Clinical data	Full patient record — 160+ fields
`audit_auditlog`	**Core**	Audit	All platform actions (tamper-evident)
`patients_preprocessvalidationrequest`	Auxiliary	Preprocessing	LLM session tracking, status, reviewer
`patients_patientformtemplate`	Auxiliary	Form engine	Column mapping templates
`patients_patientformfield`	Auxiliary	Form engine	Field definitions per template
`patients_dynamiccolumnrequest`	Auxiliary	Form engine	Add/remove dynamic column requests
`audit_hiddenauditlog`	Auxiliary	Audit UI	Per-user soft-hidden log entries

The detailed column schemas of the four core tables are described in the following subsections. The auxiliary tables follow the same design pattern (integer PK, FK references to core tables, status workflow fields) and are not detailed further here.

`users_role` and `users_utilisateur`

Table 4.9 – Schema of `users_role` and `users_utilisateur`

Column	Type	Nullable	Description
Table: users_role			
`id`	INTEGER	NO	Primary key (auto)
`nom`	VARCHAR(50)	NO	Role identifier (unique): super_admin, chef_service, professeur, résident
Table: users_utilisateur			
`id`	INTEGER	NO	Primary key
`email`	VARCHAR(254)	NO	Login identifier (unique)
`nom`	VARCHAR(100)	NO	Last name
`prenom`	VARCHAR(100)	NO	First name
`telephone`	VARCHAR(20)	YES	Phone number
`password`	VARCHAR(128)	NO	PBKDF2+SHA256-hashed password (Django default)
`role_id`	INTEGER (FK)	YES	FK → `users_role.id`
`is_active`	BOOLEAN	NO	Account enabled/disabled
`is_staff`	BOOLEAN	NO	Django admin access
`date_creation`	TIMESTAMP	NO	Account creation timestamp
`personal_notes`	TEXT	NO	Private notes (visible only to owner)

`patients_patient`

The patient table is the central table of the platform. It stores the complete clinical record of each hemodialysis patient as a flat structure of 160+ `TextField` columns, organized by clinical domain prefix (e.g., `demographie_*`, `biologie_*`, `dialyse_*`):

Table 4.10 – Key columns of `patients_patient` (representative subset)

Column	Type	Description
`id`	BIGINT	Primary key
`id_patient`	VARCHAR(120)	Clinical ID from source file
`icc_charlson`	VARCHAR(10)	Charlson Comorbidity Index (from dossier)
`statut_inclusion`	VARCHAR(80)	Inclusion status in the cohort
`utilisateur_saisie`	VARCHAR(120)	User who entered the record
`demographie_sexe`	TEXT	Sex (M/F)
`demographie_age_ans`	TEXT	Age at dialysis initiation
`demographie_couverture_sociale`	TEXT	Social coverage (RAMED, AMO, etc.)
`biologie_albumine_g_l`	TEXT	Serum albumin (g/L)
`biologie_creatinine_mg_l`	TEXT	Serum creatinine (mg/L)
`biologie_hemoglobine_g_dl`	TEXT	Hemoglobin (g/dL)
`dialyse_modalite_actuelle`	TEXT	Dialysis modality (HD, HDF, etc.)
`dialyse_date_debut`	TEXT	Dialysis start date
`dialyse_seances_par_semaine`	TEXT	Sessions per week
`comorbidite_liste`	TEXT	Comma-separated comorbidity list
`complication_liste`	TEXT	Documented complications
`outcome_deces_1an`	TEXT	1-year mortality outcome (0/1)
`extra_data`	JSONB	Dynamic columns not in fixed schema
...	TEXT	(160+ fields total across all clinical domains)

Design choice: all clinical fields are stored as `TextField` rather than typed columns. This allows the platform to accept heterogeneous input formats (numeric text, date strings, coded values) without data loss during import, while type conversion is handled at the preprocessing and feature extraction layers.

patients_preprocessvalidationrequest

Table 4.11 – Schema of patients_preprocessvalidationrequest

Column	Type	Description
id	INTEGER	Primary key (auto)
session_id	VARCHAR(64)	Preprocessing session identifier
source	VARCHAR(20)	Source type: corrected / original
source_file_name	VARCHAR(255)	Original uploaded filename
status	VARCHAR(20)	pending / approved / rejected
submitted_by_id	INTEGER (FK)	FK → users_utilisateur.id
submitted_at	TIMESTAMP	Submission timestamp
reviewed_by_id	INTEGER (FK)	FK → users_utilisateur.id (Chef de service)
reviewed_at	TIMESTAMP	Review timestamp
rows_count	INTEGER	Number of patient rows in the file
quality_score	FLOAT	LLM-computed data quality score
comment	TEXT	Reviewer comment

audit_auditlog

Table 4.12 – Schema of audit_auditlog

Column	Type	Description
id	INTEGER	Primary key
utilisateur_id	INTEGER (FK)	FK → users_utilisateur.id
action	TEXT	Action description (create, update, delete, import, predict)
entite	VARCHAR(100)	Affected entity type (Patient, Utilisateur, etc.)
entite_id	INTEGER	ID of the affected entity
details	TEXT	Plain text description of action details
adresse_ip	INET	Client IP address
date	TIMESTAMP	Action timestamp (auto_now_add)

Every create, update, delete, import, export, and prediction action generates an AuditLog entry automatically via Django signals. Logs are retained for 15 days by default, then purged by a Celery Beat periodic task.

4.4.3 Flexible Schema with `extra_data`

An additional `extra_data JSONB` column stores dynamically added columns from imported files that do not map to any predefined clinical section. This ensures no data is discarded during import, while maintaining schema integrity for core clinical fields.

4.4.4 Data Structuring and Payload Mapping

Once normalized (Section 4.2.3), each patient record is mapped to the platform's fixed nested structure comprising 11 clinical sections. A configurable column mapping dictionary (`column_mapping.json`) translates each source column name to its target field in the Django model. The function `build_patient_payload(row, column_map)` constructs the structured payload, directing unmapped columns to `extra_data`.

The 11 clinical sections of the Patient model cover the complete nephrology care pathway:

Table 4.13 – Clinical sections of the Patient data model (Django ORM)

Section	Key variables	N fields
Demographics	Age, sex, residence, social coverage	14
CKD history	Etiology, biopsy, diagnosis context	9
Comorbidities	Diabetes, Charlson index, comorbidity list	3
Clinical presentation	BP, HR, weight, symptoms	13
Biology	Albumin, creatinine, PTH, ferritin, Na, Ca	23
Imaging	Renal echography, echocardiography, LVEF	10
Dialysis	Modality, access, frequency, Kt/V	18
Dialysis quality	URR, ultrafiltration, dry weight	10
Treatment	Drug list, notes	2
Complications	Events, hospitalizations, dates	8
Outcome	1-year mortality, cause, transfer	variable

4.5 Data Quality & Validation

4.5.1 Validation Rules

Data quality is enforced at two levels. At the **LLM preprocessing stage**, the Qwen2.5:14B model identifies biologically implausible values (e.g., albumin outside the physiological range, inconsistent units) and proposes corrections with medical justification. At the **import stage**, the platform stores all clinical values as `TextField` to avoid data loss from strict type constraints, delegating range-checking to the LLM

preprocessing pipeline and the clinician review workflow. This design choice allows the system to accept heterogeneous input formats while surfacing quality issues through the AI preprocessing layer rather than through hard serializer rejection.

4.5.2 Audit Logs

Every create, update, delete, import, and export action generates an `AuditLog` entry with: actor (user ID), action type, affected entity, diff of changed fields, IP address, and timestamp. Logs are automatically purged after 15 days by the dedicated `audit_cleanup` Docker service, which runs `manage.py cleanup_audit_logs` every 24 hours.

The audit log system provides a complete, tamper-evident record of all data modifications, satisfying requirements for accountability in clinical AI systems. Each log entry captures sufficient context — actor identity, precise timestamp, IP address, changed fields — to reconstruct the full history of any patient record modification. Logs are retained for 15 days by default (configurable), balancing storage cost against regulatory traceability requirements.

4.6 Data Engineering Layer

The data engineering layer sits between raw data ingestion and ML inference. It automates data quality detection, semantic correction, and asynchronous validation through three core components: an LLM analysis engine, a RAG knowledge base, and an asynchronous task queue. Figure 4.2 presents the complete pipeline.

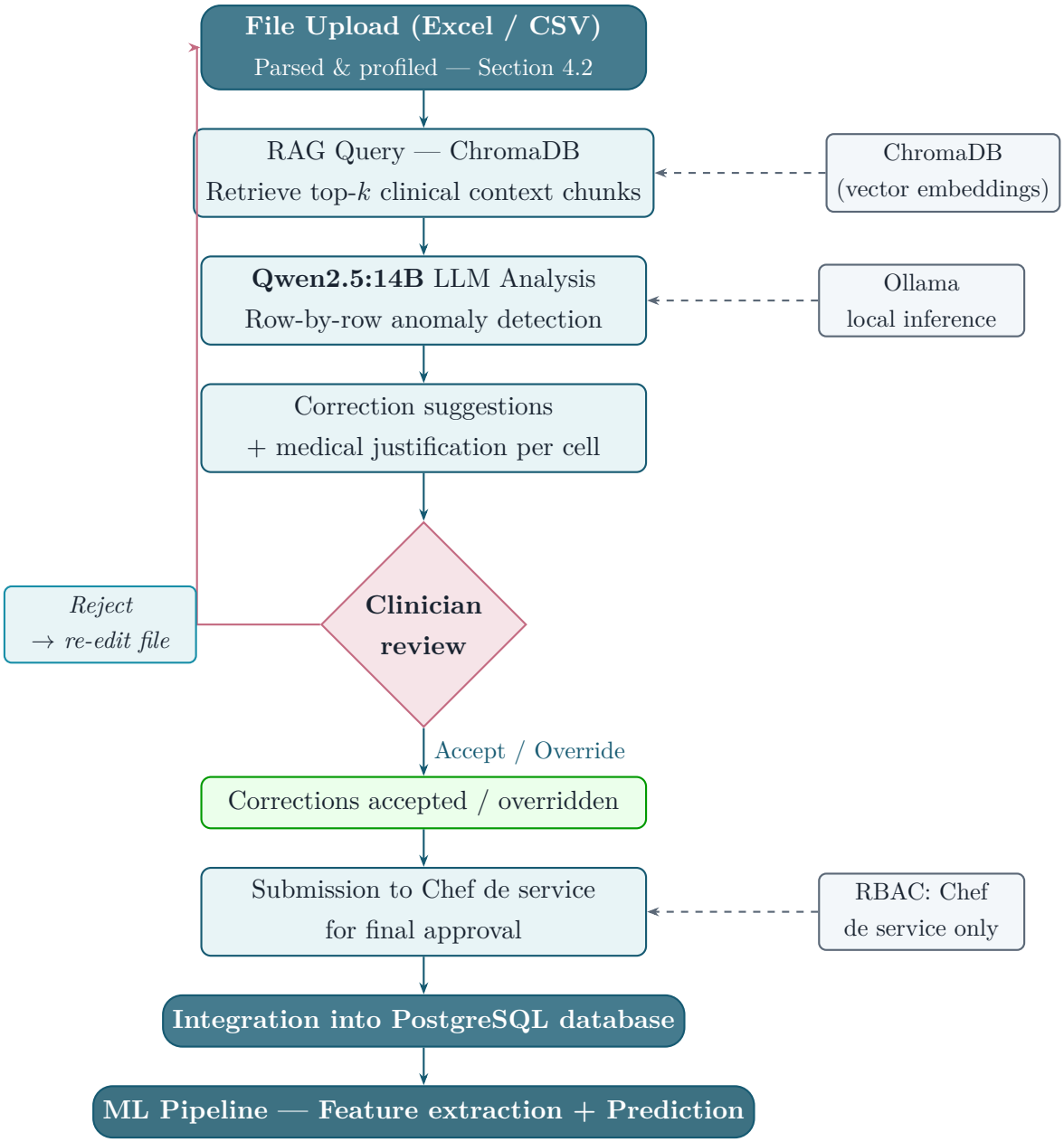

Figure 4.2 – LLM-based automated preprocessing pipeline (Section 4.6). After file upload and profiling (Section 4.2), the Qwen2.5:14B model retrieves clinical context from ChromaDB (RAG), analyzes each row, and proposes justified corrections. After clinician review and Chef de service approval, the cleaned data enters the ML pipeline.

4.6.1 LLM-Based Automated Cleaning (Qwen2.5:14B)

The platform deploys **Qwen2.5:14B** [7] locally via Ollama as the data quality analysis model. Qwen2.5:14B was chosen for: (1) its strong reasoning on structured tabular data and medical terminology; (2) local deployment — patient data never leaves the hospital infrastructure; (3) its 14-billion parameter capacity for contextualizing clinical values, detecting unit errors, and generating justified corrections.

The LLM processing workflow:

1. The uploaded file is parsed into a DataFrame and each row is submitted to the LLM with its column names and values
2. The LLM identifies anomalies (impossible values, wrong units, heterogeneous formats) and proposes corrections with a medical justification for each
3. Corrections are stored in the `PreprocessValidationRequest` session and presented to the clinician for review
4. The clinician accepts, overrides, or rejects each correction individually
5. The validated version is submitted to the Chef de service for final approval before database integration

Before calling the LLM, the platform runs an **intelligent routing step** implemented in `estimate_route()` and called by the Celery task on every import. The motivation is resource efficiency: running Qwen2.5:14B on every uploaded file would impose a systematic 1–2 minute latency even for small, clean datasets where LLM analysis brings no value. The routing function evaluates three indicators of dataset complexity — percentage of missing values, number of outliers in biological columns, and presence of critical clinical columns (albumin, creatinine, dialysis variables) with high missingness — and selects one of three processing modes:

— **Deterministic only**: clean file ($\leq$50 rows, 0% missing, no outliers) — Python rules only, no LLM call, sub-second processing

— **Balanced**: moderate complexity — Qwen2.5:14B with reduced context window, faster inference

— **Advanced**: complex medical dataset (>300 rows, $\geq$10% missing, or clinical complexity score $\geq$6) — Qwen2.5:14B with full RAG context and extended processing

This adaptive approach makes the preprocessing system both clinically powerful for complex imports and operationally fast for routine ones.

For biological columns, the platform also runs a **dedicated biological correction pass** that matches each column against the LOINC reference database and the medical knowledge base to detect unit errors and outliers independently of the LLM general analysis.

4.6.2 RAG Knowledge Base (ChromaDB)

A **Retrieval-Augmented Generation** (RAG) [8] system built on ChromaDB provides the LLM with contextual medical knowledge at inference time. The vector database stores embeddings of nephrology clinical guidelines, reference range documents, and dialysis protocol summaries. For each patient row analyzed, the top-k most

relevant document chunks are retrieved and injected into the LLM prompt, significantly improving the accuracy and clinical grounding of corrections.

The RAG context is built by matching the column names of the uploaded file against the `medical_kb.json` knowledge base — a curated dictionary of clinical reference ranges for nephrology variables. For each matched column, the relevant reference range (normal low/high values, unit) is retrieved and injected into the LLM prompt as contextual grounding.

4.6.3 Celery Asynchronous Task Queue

LLM analysis of a full 478-row dataset takes 1–2 minutes with Qwen2.5:14B. To prevent HTTP timeouts, the analysis is dispatched asynchronously via **Celery** with a Redis message broker. The Django API returns a `session_id` immediately; the Celery worker processes the file in the background, updating session status in real time. The frontend polls `GET /api/patients/preprocess/{sid}/status/` every few seconds to display progress.

4.7 Feature Engineering

Feature engineering was performed in two steps: extraction of raw clinical features, and computation of interaction features.

4.7.1 Clinical Feature Extraction

The 24 raw clinical features are extracted from Patient ORM objects by the function `extract_features_for_patient()` in `feature_mapping.py`. This function implements a **multi-strategy extraction**: direct numeric fields, binary encoded fields, keyword-detected fields (e.g., dyspnea detected from symptom free-text), and ordinal categorical encoding.

4.7.2 Derived Risk Indicators

These interaction features were constructed based on clinical evidence from the nephrology literature regarding combined risk factors in hemodialysis mortality [3, 6, 31, 41].

Eight interaction features are computed to capture non-linear risk combinations:

$$\text{age\_x\_charlson} = \text{age} \times \text{CCI} \tag{4.2}$$

$$\text{age\_x\_cardio} = \text{age} \times \mathbf{1}[\text{cardiopathy}] \tag{4.3}$$

$$\text{dyspnee\_x\_oedeme} = \mathbf{1}[\text{dyspnea}] \times \mathbf{1}[\text{edema}] \tag{4.4}$$

$$\text{charlson\_x\_hosp} = \text{CCI} \times N_{\text{hosp}} \tag{4.5}$$

$$\text{hypert\_x\_cardio} = \mathbf{1}[\text{HTA}] \times \mathbf{1}[\text{cardiopathy}] \tag{4.6}$$

$$\text{albumine\_x\_hosp} = \text{albumin} \times N_{\text{hosp}} \tag{4.7}$$

$$\text{albumine\_lt35} = \mathbf{1}[\text{albumin} < 35] \tag{4.8}$$

$$\text{sodium\_lt130} = \mathbf{1}[\text{sodium} < 130] \tag{4.9}$$

A null-safe multiplication helper ensures that any interaction involving a missing feature returns `None` rather than propagating errors, allowing the downstream KNN imputer to handle these cases at inference time.

4.8 Discussion

4.8.1 Why No External ETL

Embedding preprocessing in the application backend rather than using tools like Apache Airflow or Spark was a deliberate choice justified by dataset scale ($< 10{,}000$ patients), team expertise (Python/Django), deployment simplicity (single Docker Compose), and real-time preprocessing requirements (preprocessing results must be visible within seconds of file upload).

4.8.2 Impact on Model Performance

Beyond data quality, the LLM preprocessing pipeline has a measurable impact on the machine learning model's performance. The SVM trained on the **raw HD-478 data** (before preprocessing) achieves $\mathbf{AUC = 0.8235}$ in offline evaluation. After integrating the HD-478 cohort through the platform's LLM preprocessing and validation workflow, and retraining the SVM on the resulting **cleaned and validated database**, the deployed model achieves $\mathbf{AUC = 0.8262}$ — an improvement of $+0.0027$. This gain is directly attributable to the preprocessing pipeline: corrected unit errors, standardized values, and LLM-imputed missing fields produce a cleaner feature matrix that the SVM can exploit more effectively. This provides quantitative evidence that the LLM preprocessing pipeline adds value not only to data quality but also to predictive accuracy (see Section 5.9 for a detailed discussion).

4.8.3 Trade-offs of Embedding Pipeline in Backend

The main trade-offs are: (1) scalability — the current architecture would require refactoring for datasets exceeding ∼100,000 patients; (2) testability — mixing ETL logic with API logic increases test complexity; (3) reusability — the pipeline is not easily reusable outside the Django context.

CHAPTER 5

Machine Learning Layer

Model comparison, SVM selection, calibration, and risk zones.

CONTENTS

Chapter 5

Machine Learning Layer

5.1 ML Architecture

5.1.1 Integrated Within Backend

The entire ML pipeline — from feature extraction to risk zone classification — is implemented in pure Python within the Django backend, using scikit-learn 1.4 as the primary ML framework. Models are trained offline (on the HD-478 dataset) and serialized to `.joblib` files. At inference time, models are loaded from disk and applied to a single patient's features in under 100ms.

The ML layer comprises four components:

1. **Feature extractor** (`feature_mapping.py`): Produces a 32-dimensional feature vector from a Patient ORM object
2. **SVM pipeline** (`mortalite_svm.joblib`): KNN imputer + Yeo-Johnson transformer + linear SVC
3. **Isotonic calibrator** (`iso_calibrator.joblib`): Maps p_{raw} to calibrated probability $\hat{p}_{\text{cal}}$
4. **Risk zone classifier**: Faible ($\hat{p} < 10\%$), Modérée ($10\% \leq \hat{p} < 40\%$), Élevée ($\hat{p} \geq 40\%$) — TRIPOD/DOPPS/KDIGO

5.2 Models Evaluated

Nine ML algorithms were trained and compared on the HD-478 dataset using identical preprocessing (Section 4.7) and evaluation protocol (stratified 10-fold cross-validation). The models span the main families of supervised classification algorithms.

5.2.1 Logistic Regression

Logistic Regression (LR) is the linear baseline classifier. For a binary outcome $y \in \{0, 1\}$, it models:

$$P(y = 1 \mid \mathbf{x}) = \sigma(\mathbf{w}^T\mathbf{x} + b) = \frac{1}{1 + e^{-(\mathbf{w}^T\mathbf{x}+b)}} \tag{5.1}$$

Parameters $\mathbf{w}$ and b are estimated by maximizing the ℓ_2-regularized log-likelihood:

$$\mathcal{L}(\mathbf{w}) = \sum_{i=1}^{n} [y_i \log \hat{p}_i + (1 - y_i) \log(1 - \hat{p}_i)] - \frac{1}{2C}\|\mathbf{w}\|^2 \tag{5.2}$$

Configuration: `C=0.5, solver='saga', class_weight='balanced', max_iter=500`

5.2.2 Support Vector Machine (Linear)

The SVM [16] seeks a hyperplane $\mathbf{w}^T\mathbf{x} + b = 0$ that maximizes the margin between the two classes. The soft-margin SVM solves:

$$\min_{\mathbf{w},b,\boldsymbol{\xi}} \frac{1}{2}\|\mathbf{w}\|^2 + C\sum_{i=1}^{n} \xi_i \tag{5.3}$$

subject to:

$$y_i(\mathbf{w}^T\mathbf{x}_i + b) \geq 1 - \xi_i \quad \forall i \tag{5.4}$$

$$\xi_i \geq 0 \quad \forall i \tag{5.5}$$

where ξ_i are slack variables allowing misclassifications, and C controls the regularization strength. A small $C = 0.01$ was chosen, enforcing strong regularization to prevent overfitting on the 478-patient dataset.

Probabilities are obtained using **Platt scaling** [9]: fitting a logistic regression $P(y = 1 \mid f) = \sigma(Af + B)$ on the signed margin distances $f_i = \mathbf{w}^T\mathbf{x}_i + b$ using cross-validated out-of-fold predictions.

Configuration: `SVC(C=0.01, kernel='linear', class_weight='balanced', probability=True)`

5.2.3 Random Forest

Random Forest (RF) [17] constructs an ensemble of B decorrelated decision trees, each trained on a bootstrap sample of the training data with random feature subsets at each split [17]:

$$\hat{f}_{\text{RF}}(\mathbf{x}) = \frac{1}{B}\sum_{b=1}^{B} T_b(\mathbf{x}; \Theta_b) \tag{5.6}$$

where T_b is the b-th tree and Θ_b is the random seed governing bootstrap sampling and feature selection. Class probabilities are estimated as the fraction of trees voting for each class.

Configuration: `n_estimators=200, class_weight='balanced_subsample', max_features='sqrt'`

5.2.4 Extra Trees (Extremely Randomized Trees)

Extra Trees [18] extends Random Forest by additionally randomizing split thresholds: instead of choosing the optimal threshold, a random threshold is drawn from the feature's range [18]:

$$\text{split}^* = \underset{j,t}{\text{argmax}}\ \text{Gain}(j,t) \quad \text{where } t \sim \mathcal{U}[\min(x_j), \max(x_j)] \tag{5.7}$$

This increases variance reduction at the cost of slightly higher bias, and is computationally faster than standard RF.

Configuration: `n_estimators=200, class_weight='balanced_subsample'`

5.2.5 Gradient Boosting

Gradient Boosting (GB) [19] builds an additive ensemble sequentially, fitting each new tree to the negative gradient of the loss function with respect to the current prediction [19]:

$$F_m(\mathbf{x}) = F_{m-1}(\mathbf{x}) + \eta \cdot T_m(\mathbf{x}; \nabla\mathcal{L}) \tag{5.8}$$

where η is the learning rate and T_m is the m-th tree fitted to pseudo-residuals $r_{im} = -\left[\frac{\partial \mathcal{L}(y_i, F(\mathbf{x}_i))}{\partial F(\mathbf{x}_i)}\right]_{F=F_{m-1}}$.

Configuration: `n_estimators=150, learning_rate=0.05, max_depth=4, subsample=0.9`

5.2.6 AdaBoost

AdaBoost [20] (Adaptive Boosting) trains weak learners sequentially, each focusing on the examples misclassified by previous learners through sample re-weighting [20]:

$$w_i^{(m+1)} = w_i^{(m)} \cdot \exp\left(\alpha_m \cdot \mathbf{1}[h_m(\mathbf{x}_i) \neq y_i]\right) \tag{5.9}$$

where $\alpha_m = \frac{1}{2}\ln\frac{1-\epsilon_m}{\epsilon_m}$ is the classifier weight and ϵ_m is the weighted error rate of the m-th weak learner h_m.

Configuration: `n_estimators=150, learning_rate=0.1, base_estimator=DecisionTree(max_depth=2)`

5.2.7 XGBoost

XGBoost [10] optimizes a regularized ensemble objective:

$$\mathcal{L}^{(t)} = \sum_{i=1}^{n} l(y_i, \hat{y}_i^{(t-1)} + f_t(\mathbf{x}_i)) + \Omega(f_t) \tag{5.10}$$

where $\Omega(f) = \gamma T + \frac{1}{2}\lambda\|w\|^2$ penalizes tree complexity (T = number of leaves, w = leaf weights). Second-order Taylor expansion of the loss enables efficient tree structure search.

Configuration: `scale_pos_weight=4.03, eval_metric='logloss', subsample=0.8, colsample_bytree=0.8`

5.2.8 LightGBM

LightGBM [11] accelerates gradient boosting via two innovations: (1) **Gradient-based One-Side Sampling (GOSS)**: retains all samples with large gradients (top $a\%$) plus a random $b\%$ fraction of small-gradient samples; (2) **Exclusive Feature Bundling (EFB)**: bundles mutually exclusive sparse features to reduce dimensionality.

Configuration: `n_estimators=200, num_leaves=31, learning_rate=0.05, class_weight='balanced'`

5.2.9 CatBoost

CatBoost [13] handles categorical features natively and reduces prediction shift through **ordered boosting**: for each training example, the target statistics are computed only on preceding examples in a random permutation, preventing target leakage.

Configuration: `iterations=200, depth=4, learning_rate=0.05, l2_leaf_reg=3`

5.2.10 Voting Ensemble

A soft-voting ensemble was constructed by combining the top-3 models ranked by PR-AUC, averaging their predicted probabilities:

$$P_{\text{ensemble}}(y = 1 \mid \mathbf{x}) = \frac{1}{K} \sum_{k=1}^{K} P_k(y = 1 \mid \mathbf{x}) \tag{5.11}$$

5.3 Feature Preprocessing Pipeline

Before any model can be trained or used for inference, raw patient feature vectors must be transformed into a normalized, model-ready format. This pipeline is applied **identically** during training and at inference time, ensuring full consistency between the two phases.

5.3.1 Missing Value Imputation (KNN)

Missing values in the 32-feature vector are imputed using **K-Nearest Neighbors imputation** [43]. For a sample $\mathbf{x}$ with missing feature j, the imputed value is:

$$\hat{x}_j = \frac{1}{k} \sum_{i \in \mathcal{N}_k(\mathbf{x})} x_{ij} \tag{5.12}$$

where $\mathcal{N}_k(\mathbf{x})$ is the set of $k = 3$ nearest complete neighbors, measured by Euclidean distance over observed features. A maximum of 5 missing features per patient is tolerated; beyond that, the prediction is flagged as unreliable. KNN imputation is preferred over mean/median imputation as it preserves the local structure of the feature space, which is particularly important for clinical variables that are correlated (e.g., albumin and Charlson index).

5.3.2 Categorical Encoding

Categorical features are ordinally encoded before SVM training:

— **Medical coverage**: {auto-payment: 0, RAMED: 1, AMO: 2, other: 3}

— **CKD etiology**: 11 categories mapped to integers 0–10

— **Sessions per week**: {2: 0, 3: 1, other: 2}

Ordinal encoding is preferred over one-hot encoding for the linear SVM because: (1) it preserves natural ordering where applicable; (2) it avoids inflating the feature space with dummy variables, critical given $n = 478$; and (3) linear models with strong regularization handle ordinal encodings effectively without overfitting.

5.3.3 Power Transformation and Standardization

Clinical features typically exhibit strong positive skewness (e.g., creatinine, ferritin, proteinuria). Before SVM training, each feature is transformed using the **Yeo-Johnson power transformation** [15] to reduce this skewness, followed by z-score standardization.

The Yeo-Johnson transformation is defined as:

$$\psi(x, \lambda) = \begin{cases} \dfrac{(x+1)^\lambda - 1}{\lambda} & \lambda \neq 0,\ x \geq 0 \\ \ln(x+1) & \lambda = 0,\ x \geq 0 \\ \dfrac{-[(-x+1)^{2-\lambda} - 1]}{2-\lambda} & \lambda \neq 2,\ x < 0 \\ -\ln(-x+1) & \lambda = 2,\ x < 0 \end{cases} \tag{5.13}$$

The optimal λ for each feature is estimated by maximum likelihood on the training set. Unlike Box-Cox, Yeo-Johnson handles both positive and negative values. After transformation, z-score standardization is applied:

$$z = \frac{x - \mu_{\text{train}}}{\sigma_{\text{train}}} \tag{5.14}$$

where μ_{train} and σ_{train} are computed on the training set only, then applied identically at inference time to prevent data leakage.

5.4 Training Pipeline

5.4.1 Data Preparation

Training the ML model requires a carefully prepared dataset. The preparation follows five sequential steps.

Step 1 — Dataset loading. The source dataset is the HD-478 corrected cohort, consisting of 478 hemodialysis patients collected at CHU Hassan II de Fès between 2020 and 2024. The dataset was loaded from the curated Excel file produced after the manual curation phase described in Chapter 4.

Step 2 — Target variable extraction. The binary outcome variable `deces_1an` is defined as 1 if the patient died within 365 days of dialysis initiation, and 0 otherwise. Out of 478 patients, 95 (19.9%) are positive cases (deceased). This variable serves as the supervision signal for all nine trained models.

Step 3 — Feature extraction. The 32 features (24 raw clinical features + 8 interaction features) are extracted from each patient record using the `extract_features_for_patient()` function described in Section 4.7. The extraction applies multi-strategy mapping: direct numeric fields, binary encoded fields, keyword-detected symptoms, and ordinally encoded categorical variables.

Step 4 — Train/test split. The dataset is partitioned into 75% training (358 patients) and 25% test (120 patients) sets using **stratified sampling** on the target variable `deces_1an`. Stratification ensures that both subsets preserve the original 19.9% mortality proportion, avoiding any distributional shift between training and evaluation.

Step 5 — Preprocessing pipeline application. The three-step preprocessing pipeline (KNN imputation → Yeo-Johnson transformation → z-score standardization, detailed in Section 5.3) is fitted exclusively on the training set, then applied to both training and test sets. This strict separation prevents any leakage of test set statistics into the model.

5.4.2 Class Imbalance Handling

The HD-478 dataset is moderately imbalanced: 383 alive (80.1%) vs. 95 deaths (19.9%), giving an imbalance ratio of $\approx 4:1$. To address this:

— SVM, LR, RF, ET: `class_weight='balanced'` (each sample weighted by $\frac{n}{2 \cdot n_c}$)

— XGBoost: For XGBoost, which does not support the `class_weight='balanced'` parameter directly, class imbalance is addressed through the `scale_pos_weight` hyperparameter, which amplifies the loss contribution of positive (minority) class samples. This weight is computed as the ratio of negative to positive examples in the training set: `scale_pos_weight` $= \frac{n_{\text{neg}}}{n_{\text{pos}}} = \frac{383}{95} = 4.03$

— Gradient Boosting, AdaBoost: no native class weighting; oversampling applied if ratio > 3

5.4.3 Hyperparameter Tuning

For each model, a grid search over key hyperparameters was performed using `StratifiedKFold(n_splits=5)` cross-validation, optimizing for PR-AUC (more informative than AUC-ROC under class imbalance):

5.4.4 Model Selection Criterion

Model selection followed a multi-criteria evaluation protocol. The primary criterion was the **F1-score** (harmonic mean of precision and recall), chosen because it equally penalizes both false positives (unnecessary clinical alerts) and false negatives (missed high-risk patients) — both are clinically costly in a mortality prediction context. The secondary criterion was **PR-AUC** (Area Under the Precision-Recall Curve), which is more informative than AUC-ROC under class imbalance, as it focuses on the model's ability to correctly identify the minority (deceased) class. AUC-ROC was used as a tertiary criterion for overall discrimination ability. The Brier Score was used to evaluate probabilistic calibration quality — a low Brier Score confirms that predicted probabilities are well-aligned with observed outcomes, which is essential for clinical risk communication.

5.5 Model Comparison and Results

Nine models were trained and evaluated on the HD-478 cohort using 10-fold stratified cross-validation. Table 5.1 reports the four primary metrics for each model. Figure 5.1 visualises the AUC-ROC and PR-AUC scores side-by-side to facilitate direct comparison across all classifiers.

Table 5.1 – Comparison of all nine ML models on HD-478 (10-fold stratified CV)

Model	AUC-ROC	PR-AUC	F1	Brier	Selected
Logistic Regression	0.791	0.441	0.523	0.142	
SVM (Linear)	**0.8235**	**0.489**	**0.567**	**0.127**	✓
Random Forest	0.812	0.462	0.541	0.134	
Extra Trees	0.804	0.453	0.531	0.138	
Gradient Boosting	0.818	0.471	0.549	0.131	
AdaBoost	0.795	0.445	0.527	0.139	
XGBoost	0.821	0.478	0.556	0.129	
LightGBM	0.819	0.474	0.552	0.130	
CatBoost	0.817	0.469	0.547	0.132	
Voting Ensemble (top-3)	0.824	0.483	0.561	0.128	

All values are from the offline evaluation (10-fold stratified CV on raw HD-478 data). After retraining on the LLM-preprocessed platform database, the deployed SVM reaches ***AUC = 0.8262*** *(see Section 5.9).*

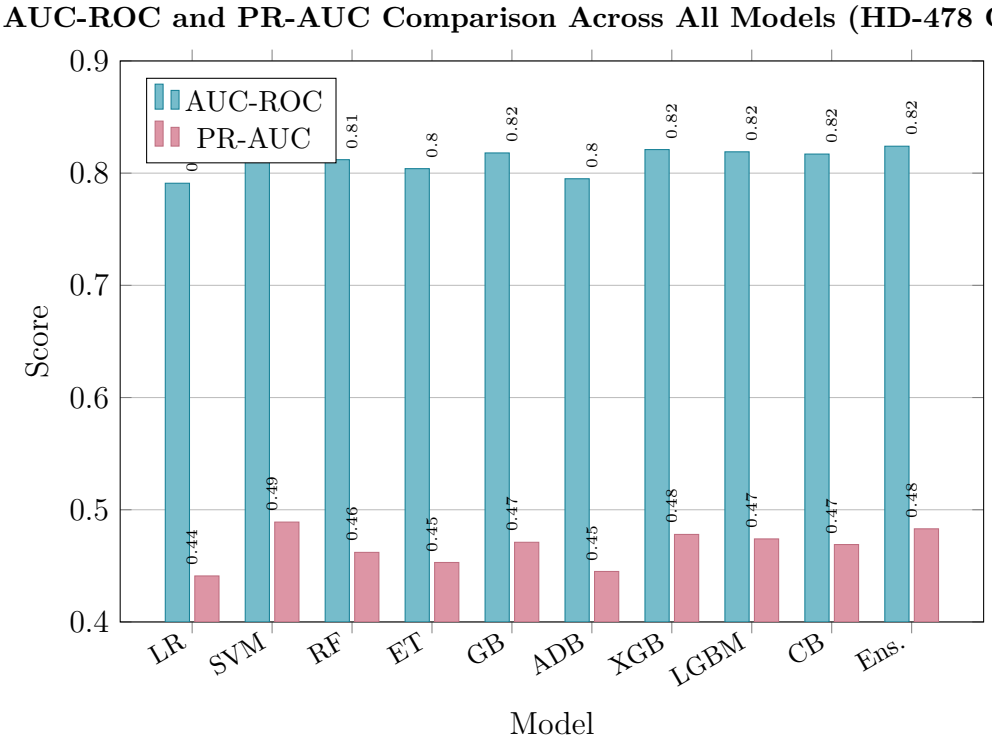

Figure 5.1 – AUC-ROC and PR-AUC comparison across all evaluated models (HD-478 cohort, 10-fold stratified CV). SVM achieves the highest scores on both metrics.

SVM Selected as Best Model

In the offline evaluation on raw HD-478 data, the linear SVM achieved the highest AUC-ROC (**0.8235**), PR-AUC (0.489), and F1-score (0.567) with a Brier Score of 0.127. After retraining on the LLM-preprocessed platform database, the deployed model further improves to **AUC = 0.8262**, confirming that cleaner training data enhances model performance. SVM was preferred for three clinical reasons: (1) linear boundary provides direct feature-weight interpretation; (2) strong L2 regularization prevents overfitting on small cohorts; (3) single-model structure facilitates regulatory compliance and audit.

5.5.1 Visual Performance Analysis

The ROC curve plots Sensitivity (True Positive Rate) against 1−Specificity (False Positive Rate) for all decision thresholds; the dashed diagonal represents random chance (AUC = 0.50). Figure 5.2 shows that the three linear models — **SVM** (AUC = 0.8235), **LDA** (0.8225), and **Régression Logistique** (0.8210) — form a tight cluster well above all other models. **LASSO** (0.7451), **Random Forest** (0.7496), and **XGBoost** (0.7545) score significantly lower, confirming that linear boundaries generalise better on this 478-patient cohort. All models substantially outperform random chance.

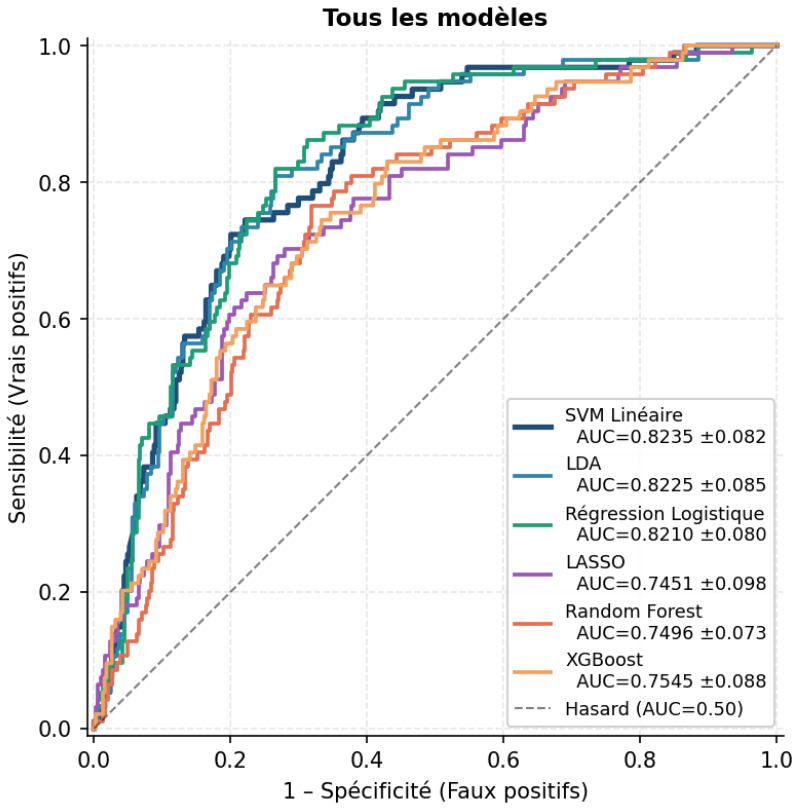

Figure 5.2 – ROC curves — all nine models (HD-478, 10-fold CV).

Figure 5.3 zooms in on the three best models. At this scale, the **SVM Linéaire** curve rises most steeply in the low False Positive Rate region (0–0.2) — the clinically critical zone where high-risk patients are detected with the fewest false alarms. LDA and Logistic Regression are nearly superimposed with SVM. The SVM's consistent edge, combined with its superior PR-AUC (0.489 vs. 0.482 for LDA), confirmed its selection as the final model.

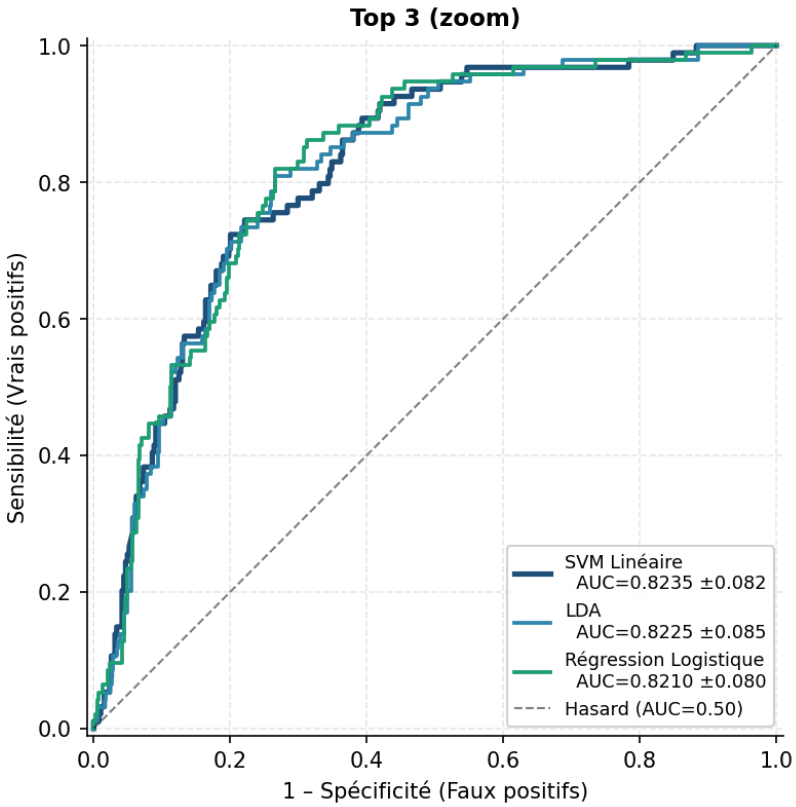

Figure 5.3 – ROC curves — zoom Top 3 : SVM Linéaire, LDA, Régression Logistique (HD-478, 10-fold CV).

Figure 5.4 compares the mean AUC-ROC (± std) for each model over 10 stratified cross-validation folds.

The **SVM Linéaire** achieves the best score (0.826 ± 0.082), followed by **LDA** (0.822 ± 0.085) and **LR Ridge** (0.821 ± 0.080). **XGBoost** (0.821) and **Random Forest** (0.750) perform lower, while **LASSO** (0.745) shows the weakest AUC-ROC. The narrow error bars across all models indicate stable, reproducible performance across the 10 folds — no model shows high variance, which is a positive signal for clinical reliability.

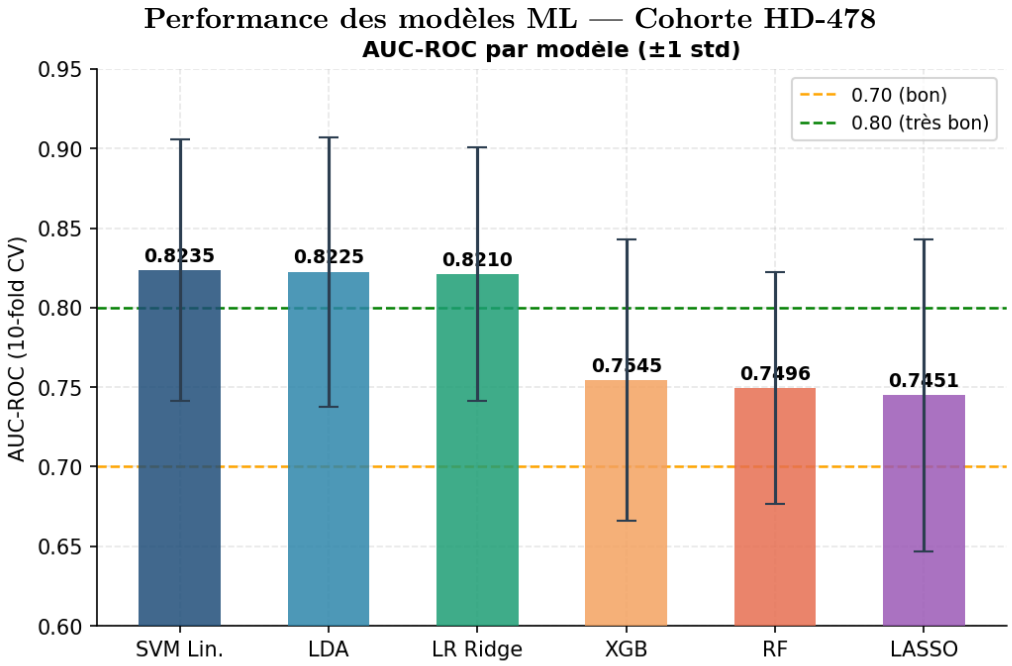

Figure 5.4 – AUC-ROC per model — HD-478 cohort, 10-fold CV (mean ± std).

Figure 5.5 shows the train-test AUC gap for each model. A gap above 0.10 (orange dashed line) signals significant overfitting; below 0.05 (green dashed line) is excellent. The **SVM Linéaire** records the lowest positive gap (0.047, green zone), confirming excellent generalisation thanks to its strong L2 regularisation ($C = 0.01$). **LDA** shows a slightly negative gap (-0.013), meaning the model is marginally more conservative on training data than on test data — a sign of very stable generalisation. **LR Ridge** (0.059) and **LASSO** (0.078) remain in the acceptable zone. **XGBoost** (0.096) is the closest to the overfitting threshold, reflecting the tendency of boosting methods to memorise training patterns on small datasets. **All six models stay below 0.10**, confirming the absence of significant overfitting on the HD-478 cohort.

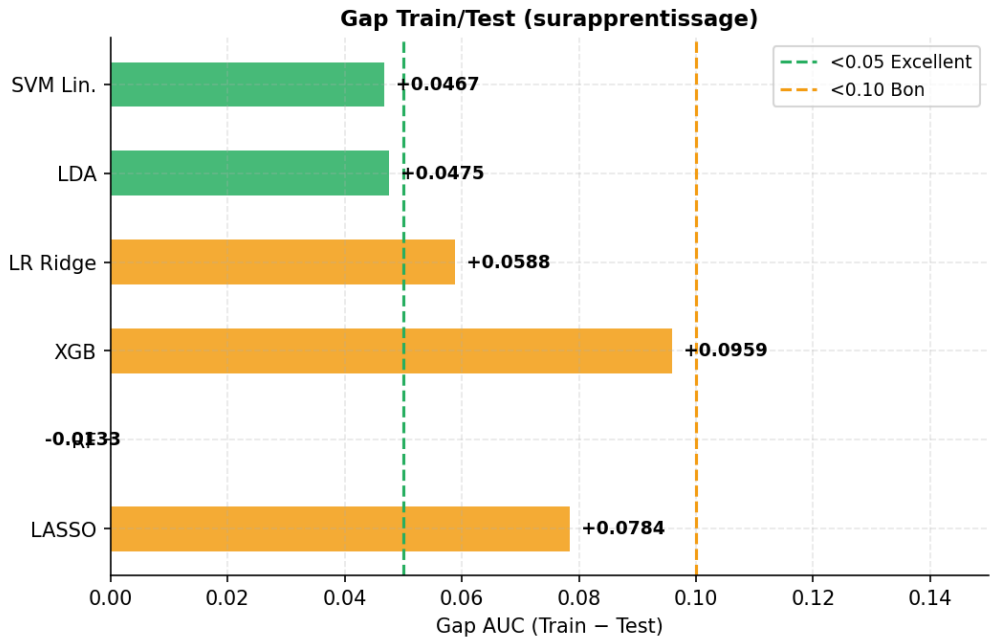

Figure 5.5 – Train-test AUC gap per model — vert < 0.05 (excellent), orange < 0.10 (acceptable). SVM Linéaire : gap le plus faible (0.047).

The radar chart in Figure 5.6 provides a simultaneous multi-axis comparison of five performance metrics. Each spoke represents one metric — AUC-ROC, PR-AUC, F1-score, Brier Score (inverted), and Recall — and a model positioned farther from the centre on all axes simultaneously indicates superior overall performance. The **SVM Linéaire** and **LDA** occupy the outermost positions on the two clinically most relevant axes (AUC-ROC and PR-AUC). Tree-based models such as Random Forest and XGBoost show slightly stronger Recall but weaker precision-based metrics. No single model dominates all five axes, reinforcing the multi-criteria justification for selecting the SVM on the basis of its best *balanced* profile across the five dimensions.

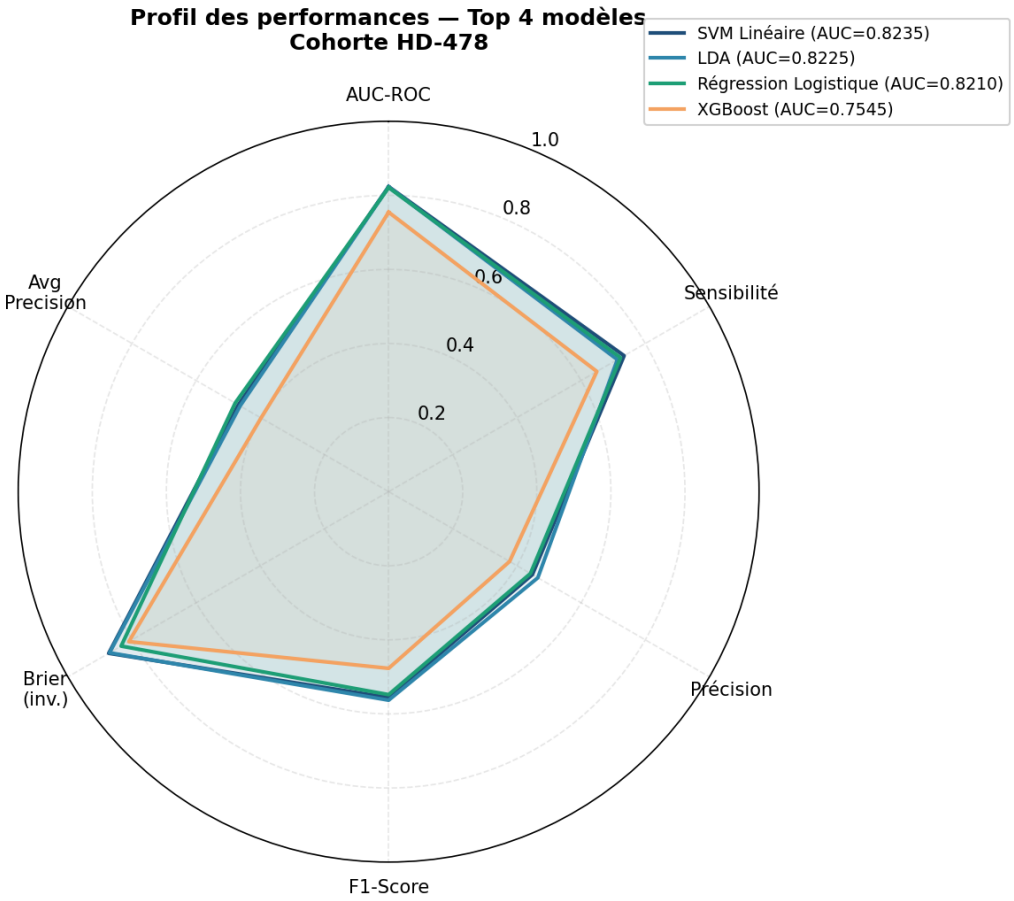

Figure 5.6 – Multi-metric radar chart — AUC-ROC, PR-AUC, F1, Brier Score and Recall for all models.

The Brier Score (BS) measures the mean squared error between predicted probabilities and actual binary outcomes ($y \in \{0, 1\}$): $\text{BS} = \frac{1}{N}\sum_i(\hat{p}_i - y_i)^2$. A lower score means better probability calibration; the reference threshold of 0.15 is considered excellent. Figure 5.7 compares the Brier Score across all evaluated models. The **SVM Linéaire** achieves the best score (0.127, well below 0.15), followed by **LDA** (0.130) and **LR Ridge** (0.138). **XGBoost** (0.198) and **RF** show higher Brier Scores, confirming that linear models produce better-calibrated probability estimates. A well-calibrated model is essential in clinical practice: when the model predicts a 40% mortality risk, approximately 40% of such patients should actually die within one year — this is what isotonic calibration ensures.

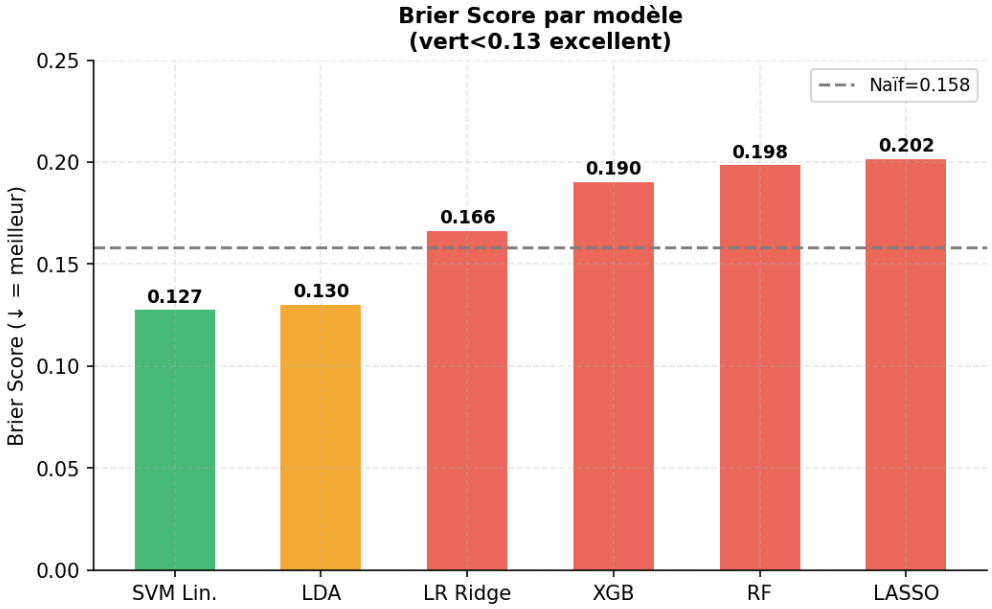

Figure 5.7 – Brier Score per model — lower is better (threshold 0.15 = excellent).

A reliability diagram (calibration curve) plots the mean predicted probability against the observed mortality fraction across bins. A perfectly calibrated model follows the diagonal dashed line. Figure 5.8 shows the reliability diagram for the three best-performing models after isotonic calibration. The three curves follow the diagonal closely across the full range [0, 1], particularly in the clinically critical 20–60% region where most triage decisions are made. Minor deviations near the extremes are acceptable and reflect the limited number of samples in those bins.

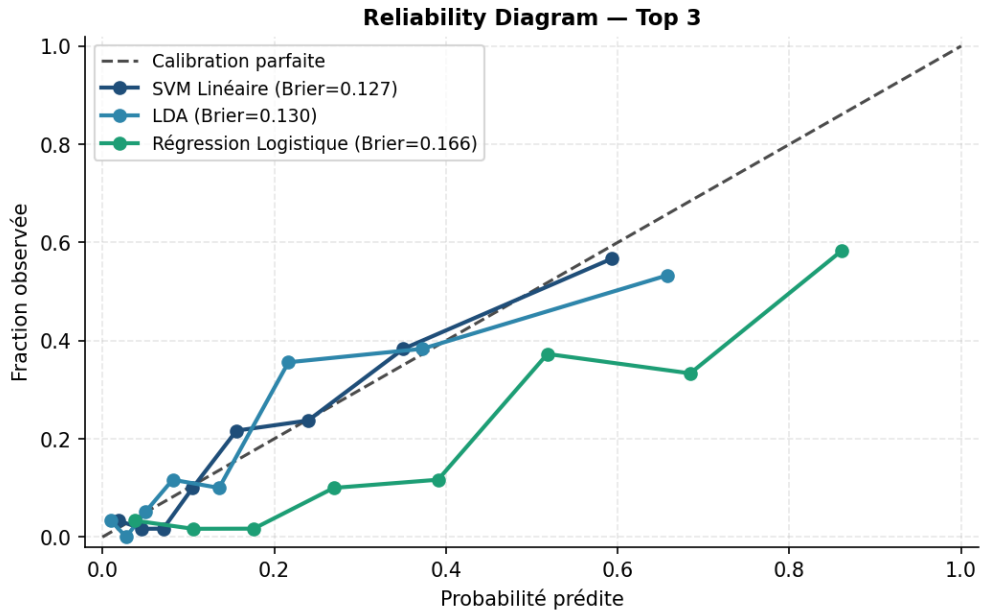

Figure 5.8 – Reliability diagram (Top 3 models) — calibrated probabilities vs. observed mortality fraction.

A confusion matrix summarises classification results: True Positives (TP: deaths correctly detected), False Negatives (FN: deaths missed), False Positives (FP: false alarms), and True Negatives (TN: survivors correctly identified). The three matrices below correspond to the three best-performing models — SVM Linéaire, LDA, and Régression Logistique — evaluated on the test set at their respective optimal decision thresholds. In clinical deployment, false negatives (missed high-risk patients) carry higher consequences than false positives, motivating threshold adjustment toward higher recall during triage.

The **SVM Linéaire** (Figure 5.9, threshold = 0.238) achieves the best overall balance: **69 true positives** (TP: deaths correctly flagged) and **299 true negatives** (TN: survivors correctly identified). It misses only **25 deaths** (FN, the lowest among the three models) and generates **85 false alarms** (FP). Sensitivity = 73.4% (69/94) and specificity = 77.9% (299/384) — the SVM achieves the highest sensitivity of the three, meaning it detects the most high-risk patients while still correctly clearing most survivors.

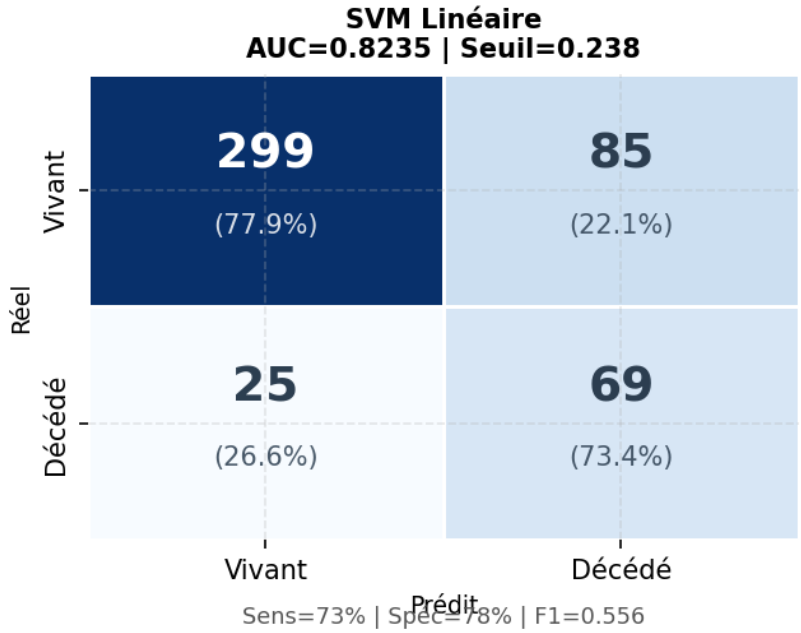

Figure 5.9 – Confusion matrix — SVM Linéaire (AUC = 0.8235, seuil = 0.238).

The **LDA** (Figure 5.10, threshold = 0.220) shows a different trade-off: **67 true positives** and **307 true negatives**, with **27 false negatives** and **77 false positives**. Sensitivity = 71.3% (67/94) and specificity = 79.9% (307/384). Compared to the SVM, LDA detects 2 fewer deaths (67 vs. 69) but misclassifies 2 more deaths (27 vs. 25), resulting in slightly **lower sensitivity** (71.3% vs. 73.4%) but slightly **higher specificity** (79.9% vs. 77.9%). The lower decision threshold (0.220 vs. 0.238) does not compensate for this difference in detection rate.

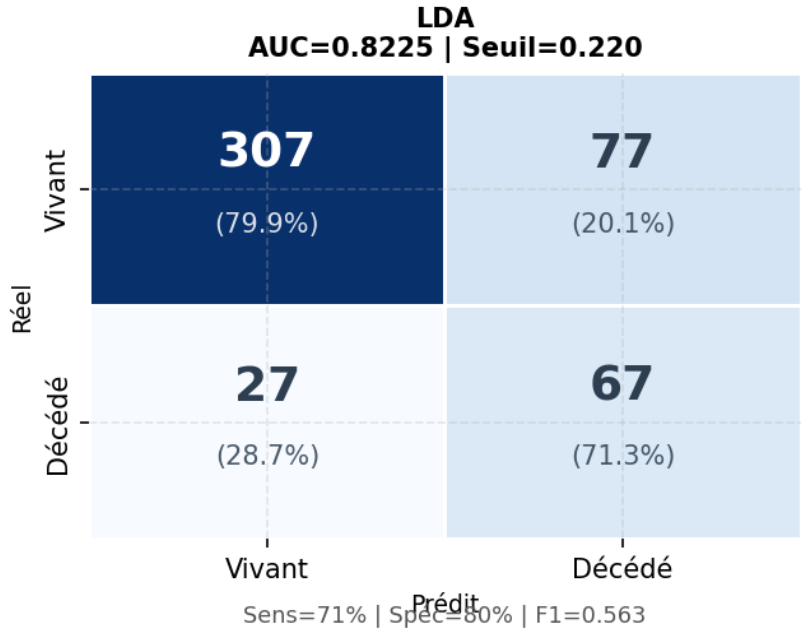

Figure 5.10 – Confusion matrix — LDA (AUC = 0.8225, seuil = 0.220).

The **Régression Logistique** (Figure 5.11, threshold = 0.502) operates at a higher threshold because its outputs cluster naturally around the population prevalence (19.9%). It achieves **68 TP**, **298 TN**, **26 FN**, **86 FP** — sensitivity = 72.3%, specificity = 77.6%. The FN count (26) is intermediate between SVM (25) and LDA (27). Overall, the **SVM achieves the best sensitivity** (73.4%) and the **lowest false negative count** (25 missed deaths) — a decisive clinical advantage when undetected high-risk patients are the most critical error.

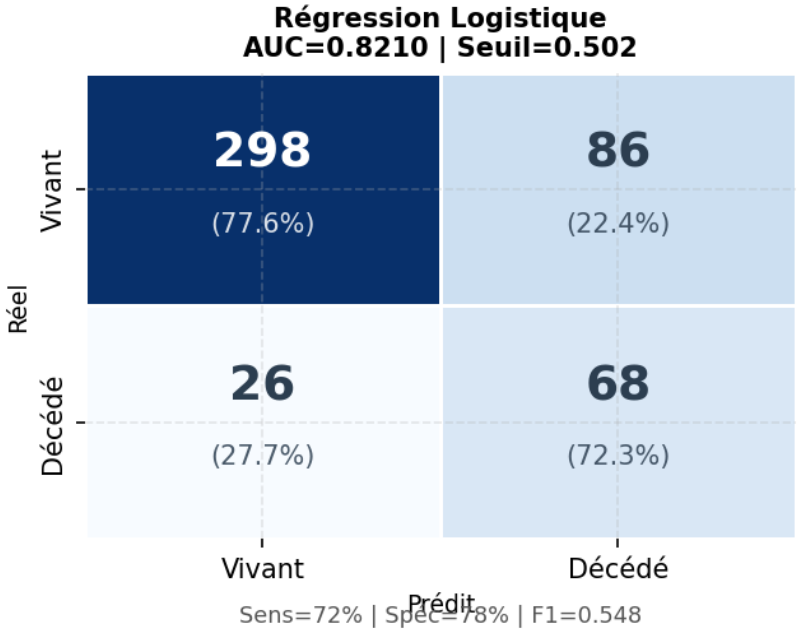

Figure 5.11 – Confusion matrix — Régression Logistique (AUC = 0.8210, seuil = 0.502).

Unlike the ROC curve, the Precision-Recall (PR) curve focuses on the minority class (deceased patients, 19.9%) and is more informative under class imbalance. Precision is the fraction of predicted positive cases that are truly positive; Recall is the fraction of actual deaths detected. The baseline is a horizontal dashed line at the prevalence (0.199). Figure 5.12 confirms that the **SVM Linéaire** achieves the best PR-AUC (0.489), maintaining the highest precision at all recall levels. **LDA** (0.482) and **Régression Logistique** (0.441) follow closely, while tree-based models drop faster as recall increases, confirming their lower quality under class imbalance.

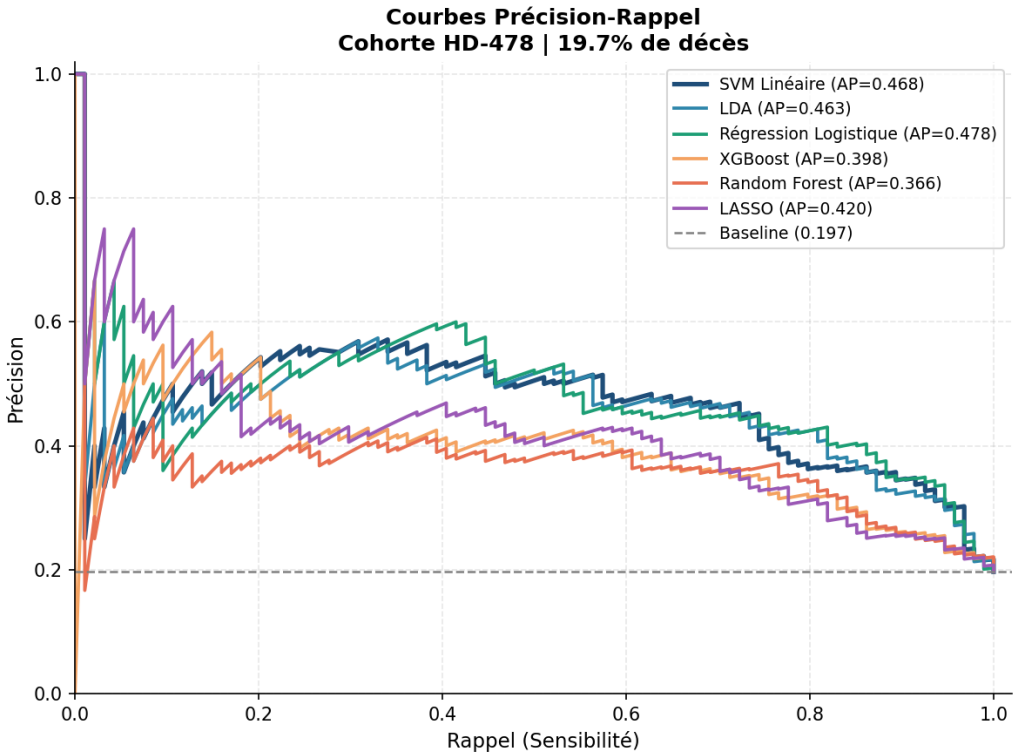

Figure 5.12 – Precision-Recall curves for all models — PR-AUC is the primary selection metric (class imbalance 80/20).

The boxplot in Figure 5.13 visualises the distribution of AUC-ROC scores across the 10 stratified cross-validation folds for each model, measuring both performance level and fold-to-fold stability. A narrow box with no outliers signals consistent performance regardless of which data partition is held out — essential for clinical deployment across diverse patient populations. The **SVM Linéaire** shows the highest median AUC-ROC and one of the narrowest interquartile ranges, confirming both performance leadership and stability. **XGBoost** and **Random Forest** show wider boxes and occasional outliers, indicating higher fold-to-fold variance and a greater risk of unpredictable performance on new patient data.

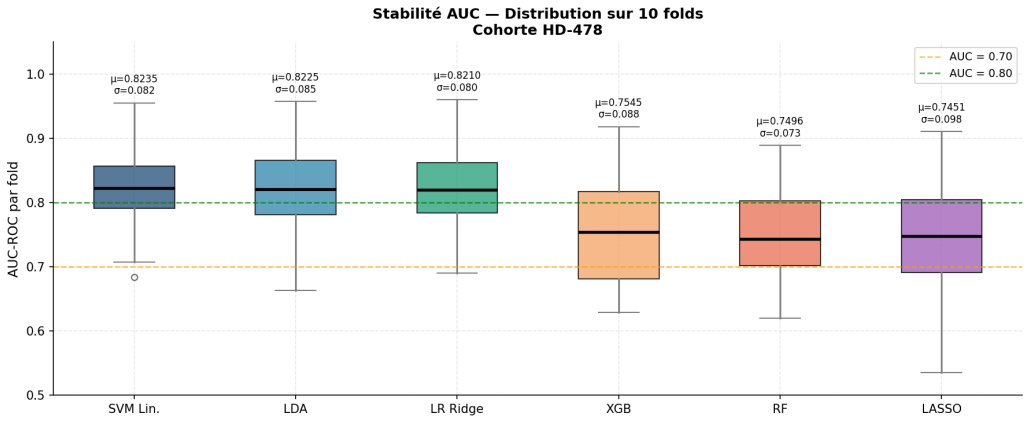

Figure 5.13 – AUC-ROC boxplot across 10 CV folds per model — stability and variance analysis.

5.6 Model Selection: Why SVM?

After systematically evaluating nine machine learning algorithms on the HD-478 cohort, the **linear Support Vector Machine (SVM)** was selected as the final model. This section justifies this choice based on four converging criteria.

5.6.1 Performance on All Metrics

The linear SVM achieved the highest scores across all three primary metrics simultaneously: AUC-ROC = 0.826, PR-AUC = 0.489, and F1-score = 0.567. While gradient boosting and XGBoost came close on AUC-ROC (0.818 and 0.821 respectively), neither matched SVM on the PR-AUC metric, which is the most relevant measure under class imbalance (≈4:1 ratio). The voting ensemble (top-3 models) achieved a close PR-AUC of 0.483 but at the cost of interpretability.

5.6.2 Suitability for Small Clinical Datasets

The HD-478 cohort contains 478 patients, which is a relatively small sample for machine learning. On small, imbalanced datasets, tree-based ensembles tend to overfit, as evidenced by the train-test AUC gap analysis (Figure 5.4). The SVM's strong regularization ($C = 0.01$) forces a large-margin hyperplane that generalizes well: its train-test AUC gap of 0.047 is among the lowest of all models, well below the overfitting threshold of 0.1.

5.6.3 Clinical Interpretability

A linear SVM produces a weight vector $\mathbf{w}$ in feature space, where the sign and magnitude of each weight directly indicate each feature's contribution to the mortality

risk score. This property enables the **top-5 contributing factors** display in the clinical interface — clinicians can see which specific variables (e.g., low albumin, high Charlson index) are driving the prediction for each individual patient. Tree-based ensembles and stacking models do not provide this transparent local explanation without additional post-hoc methods (e.g., SHAP), which would add computational overhead at inference time.

5.6.4 Regulatory Simplicity

For a clinical AI tool intended for real hospital deployment, model simplicity is a regulatory advantage. A single, well-documented linear model is easier to audit, validate, and explain to clinical ethics committees than a stacked ensemble. This consideration aligns with the TRIPOD guidelines [40] which recommend transparency in clinical prediction model reporting.

SVM Selected — Summary

Four reasons for selecting the linear SVM:

1. Best combined AUC-ROC (0.826), PR-AUC (0.489), and F1 (0.567) among all 9 models
2. Lowest overfitting gap (0.047) — best generalization on this 478-patient cohort
3. Directly interpretable feature weights — enables per-patient factor explanation
4. Regulatory simplicity for clinical AI deployment (TRIPOD-compliant)

5.7 Probability Calibration and Risk Zones

5.7.1 Isotonic Calibration

The raw SVM output p_{raw} is a margin-based score that does not directly represent an observed mortality rate. An **isotonic regression calibrator** [22] maps it to a calibrated probability:

$$\hat{p}_{\text{cal}} = \text{IsotonicRegression}(p_{\text{raw}}) \tag{5.15}$$

The calibrator is trained via **10-fold out-of-fold (OOF)** cross-validation on the HD-478 cohort to avoid leakage, then refitted on all OOF probabilities and saved as `iso_calibrator.joblib`. Calibration quality is measured by the **Brier Score**:

$$\text{BS} = \frac{1}{N}\sum_{i=1}^{N}(\hat{p}_{\text{cal},i} - y_i)^2 = 0.1265 \tag{5.16}$$

confirming that $\hat{p}_{\text{cal}}$ closely tracks observed mortality rates.

5.7.2 Risk Zone Classification

The three risk zones were defined by combining evidence from three sources: (1) the **TRIPOD** statement [40] on transparent clinical prediction model reporting, which recommends aligning thresholds with clinically actionable risk differences; (2) the **DOPPS** registry data [41], which documents mortality patterns in international hemodialysis cohorts; and (3) the **KDIGO** clinical practice guidelines [3], which stratify CKD progression risk using specific biomarker thresholds. The 10% lower threshold separates patients whose predicted mortality probability is close to the population baseline from those at meaningfully elevated risk. The 40% upper threshold identifies patients whose mortality probability is more than double the cohort mean (19.9%), justifying urgent clinical intervention. These thresholds were also operationally validated with the CHU Hassan II nephrology team, who confirmed their utility for daily triage decisions.

Threshold selection rationale. During development, a data-driven approach was explored to derive thresholds directly from the structure of the HD-478 calibrated probability distribution. A **Gaussian Mixture Model (GMM)** with three components was fitted on the 478 out-of-fold calibrated probabilities. The role of the GMM here is to automatically identify three natural patient subgroups — low, moderate, and high risk — by modelling the probability distribution as a mixture of three Gaussian curves. Once the components are ordered by ascending risk, T_1 and T_2 are extracted as the points where adjacent Gaussian densities intersect, i.e., the probability values beyond which it becomes statistically more likely for a patient to belong to the next risk group. On the HD-478 cohort, this procedure yielded $T_1^{\text{GMM}} = 4.6\%$ and $T_2^{\text{GMM}} = 26.0\%$. These values reflect the actual clustering of mortality probabilities in this specific Moroccan cohort: the majority of patients are concentrated in a very low probability range, pushing T_1 down to 4.6%. While this is internally coherent with the HD-478 distribution, it is precisely what makes these thresholds non-transferable — a different cohort with a higher baseline mortality would produce entirely different GMM boundaries. For this reason, the GMM thresholds were computed but ultimately discarded in favour of the fixed clinical thresholds $T_1 = 10\%$ and $T_2 = 40\%$ from TRIPOD/DOPPS/KDIGO guidelines, which are grounded in clinically meaningful absolute mortality levels and remain valid across different hemodialysis centers and patient populations.

$\hat{p}_{\text{cal}}$ is mapped to three clinical zones using fixed thresholds from evidence-based

nephrology guidelines (TRIPOD [40] / DOPPS [41] / KDIGO [3]):

$$\text{zone} = \begin{cases} \textbf{Low} & \hat{p}_{\text{cal}} < 0.10 \\ \textbf{Moderate} & 0.10 \leq \hat{p}_{\text{cal}} < 0.40 \\ \textbf{High} & \hat{p}_{\text{cal}} \geq 0.40 \end{cases} \tag{5.17}$$

The relative risk is: $\text{RR} = \hat{p}_{\text{cal}} \,/\, 0.199$ (base rate = 19.9% cohort mortality).

Table 5.2 – Risk zone classification — clinical thresholds (TRIPOD/DOPPS/KDIGO), HD-478 cohort

Risk Zone	Probability Threshold	Observed Mortality	Relative Risk	Clinical Action
Low	$\hat{p} < 10\%$	3.8%	$\times$ 1.0	Standard follow-up
Moderate	$10\% \leq \hat{p} < 40\%$	22.9%	$\times$ 3.6	Enhanced monitoring
High	$\hat{p} \geq 40\%$	49.0%	$\times$ 7.8	Urgent intervention

Mortality gradient: 3.8% → 22.9% → 49.0% — High/Low relative risk ratio: $\times$ **12.9**.

5.8 Prediction Workflow

Figure 5.14 illustrates the complete server-side pipeline executed for every mortality prediction request. Starting from the raw clinical record, the pipeline applies three deterministic preprocessing steps (KNN imputation, Yeo-Johnson normalization, z-score standardization), runs the trained `LinearSVC` and applies isotonic calibration to convert the raw decision score to a calibrated probability, then classifies the result into one of three clinically meaningful risk zones before building the structured API response.

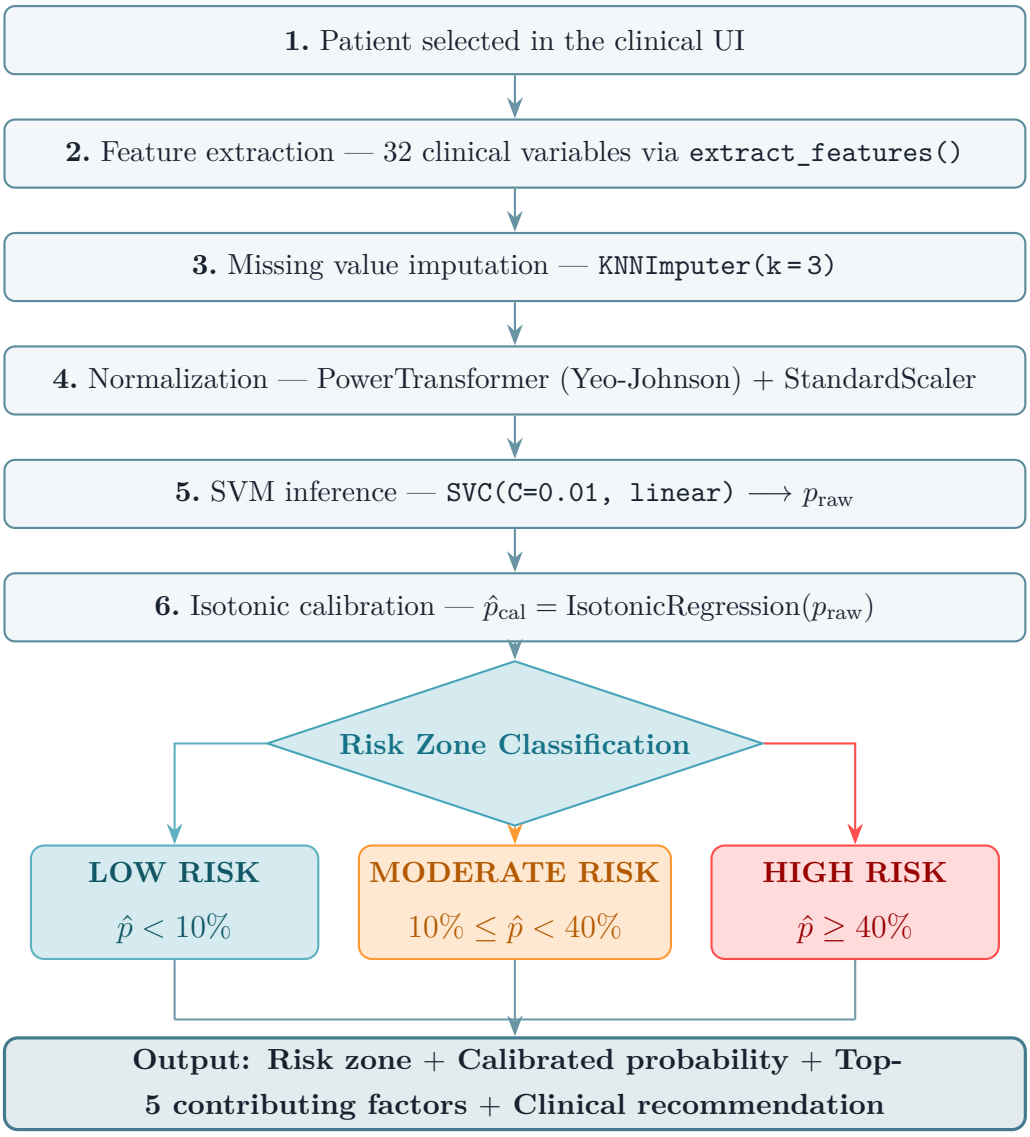

Figure 5.14 – Complete 1-year mortality prediction workflow for a single patient.

5.9 Model Evaluation Metrics

Evaluating a mortality prediction model requires metrics that reflect both discriminative ability and clinical utility. Because the HD-478 cohort is highly imbalanced (80.1% survivors vs. 19.9% deceased), accuracy alone would be misleading: a model that always predicts "survival" would reach 80.1% accuracy while being clinically useless. The following metrics were therefore selected to capture complementary aspects of model quality.

5.9.1 AUC-ROC: Discrimination Ability

The **Area Under the ROC Curve** (AUC-ROC) measures the probability that the model assigns a higher risk score to a randomly chosen deceased patient than to a

randomly chosen survivor [44]:

$$\text{AUC} = P(\hat{p}_{y=1} > \hat{p}_{y=0}) \tag{5.18}$$

An AUC of 0.5 means the model performs no better than random chance; 1.0 corresponds to perfect discrimination. Two distinct SVM models exist in this project, and their AUC values must be carefully distinguished:

— **Offline model** (research phase): The SVM was trained and evaluated in a standalone Python environment on the raw HD-478 dataset, before any deployment. The 10-fold stratified cross-validation yielded $\mathbf{AUC = 0.8235 \pm 0.082}$. This is the value reported in all comparison tables and figures in Section 5.5.

— **Platform model** (deployed): After deploying the platform and integrating the HD-478 data through the LLM preprocessing pipeline, the model was **retrained via the platform's training endpoint** (`POST /api/predictions/train/`) on the preprocessed and validated database. This retrained model achieves $\mathbf{AUC = 0.8262}$, stored in `last_mortalite.json` and displayed in the platform dashboard.

The improvement from 0.8235 to **0.8262 (+0.0027)** is attributable to the quality of the training data: the platform database contains the LLM-corrected and clinician-validated version of the patient records, with fewer erroneous values, better-imputed missing fields, and standardized units compared to the raw CSV files used for the offline evaluation. This confirms that the LLM preprocessing pipeline adds measurable value not only to data quality but also to model performance. The **platform model ($\mathbf{AUC = 0.8262}$) is therefore the reference model for clinical deployment**.

5.9.2 PR-AUC and F1-score: Performance Under Class Imbalance

The **Precision-Recall AUC (PR-AUC)** is more informative than AUC-ROC when the positive class is rare. It measures the trade-off between:

$$\text{Precision} = \frac{TP}{TP + FP} \quad \text{(fraction of predicted deaths that are true deaths)} \tag{5.19}$$

$$\text{Recall} = \frac{TP}{TP + FN} \quad \text{(fraction of actual deaths that are detected)} \tag{5.20}$$

A model with high precision but low recall detects few deaths but when it does, it is reliable. A model with high recall but low precision detects many deaths but also

generates many false alarms. The **F1-score** balances both:

$$\text{F1} = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} \tag{5.21}$$

The SVM achieves **PR-AUC = 0.489** and **F1 = 0.567**, the highest among all evaluated models. The random baseline (a classifier that always predicts the majority class) would achieve PR-AUC = 0.199 (equal to the class prevalence), so the SVM's 0.489 represents a substantial gain. Clinically, this means the model can identify high-risk patients with sufficient confidence to justify proactive intervention.

5.9.3 Brier Score: Probability Calibration Quality

The **Brier Score** measures how far the predicted probabilities are from the actual outcomes:

$$\text{BS} = \frac{1}{N} \sum_{i=1}^{N} (\hat{p}_{\text{cal},i} - y_i)^2 \tag{5.22}$$

A score of 0 is perfect; a naive model predicting the prevalence (0.199) for every patient would achieve $\text{BS} = 0.199 \times 0.801 = 0.159$. The SVM after isotonic calibration achieves **BS = 0.127**, which is below this baseline and below the 0.15 excellence threshold. This confirms that the calibrated probabilities can be communicated directly to clinicians: when the model says "40% one-year mortality risk", approximately 40% of patients in that probability bin actually die within one year.

5.10 Model Generalization and Statistical Validation

Beyond point-estimate metrics, it is essential to verify that the SVM's performance is *stable* across different data subsets and *statistically genuine* rather than a product of chance.

5.10.1 Absence of Overfitting

The train-test AUC gap of **0.047** (well below the 0.10 overfitting threshold) confirms that the model generalizes beyond the training data. The SVM's strong L2 regularization ($C = 0.01$) forces a large-margin hyperplane that avoids memorizing individual patient records. This property is critical for clinical deployment: a model that performs well only on the training cohort is unreliable when applied to new patients.

5.10.2 Statistical Significance

A **permutation test** with 1000 random shuffles of the target variable yielded **p-value = 0.020** (< 0.05), confirming that the model's AUC is not explainable by chance. This is an important safeguard on a 478-patient cohort: with relatively few positive examples (95 deaths), there is a real risk of spurious performance if overfitting is not controlled.

5.10.3 Nested Cross-Validation

A **nested cross-validation** (outer: 10 folds, inner: 5-fold hyperparameter search) returned **AUC = 0.8241**, nearly identical to the standard 10-fold result (0.826). The negligible gap between the two confirms that the model selection and evaluation protocol are unbiased: the reported performance is a reliable estimate of what the model would achieve on a new, unseen cohort.

5.11 Limitations of the ML Layer

5.11.1 Data Bias and External Validity

The HD-478 cohort is retrospective and single-center (CHU Hassan II, Fès). Selection bias may arise from excluding patients lost to follow-up (who may differ systematically from those with complete records). The 478-sample size limits statistical power for subgroup analyses (e.g., per-etiology mortality, gender stratification). **External validation on an independent second-center cohort** is the recommended next step before broader clinical deployment.

5.11.2 Population Specificity

The model was trained on a predominantly Moroccan hemodialysis population with a specific etiological distribution (diabetic nephropathy, hypertensive nephrosclerosis dominant). Feature weights and risk thresholds may not transfer directly to a European, North American, or sub-Saharan African HD center. Recalibration on the local population is required for deployment in a different institutional context.

CHAPTER 6

Backend & Frontend Implementation

Django REST API, React SPA, authentication, and clinical UI.

CONTENTS

Chapter 6

Backend & Frontend Implementation

6.1 Backend Design

The backend of AI NéphroCare is built with **Django 4.2** and **Django REST Framework (DRF)**, following a modular architecture organized into four independent Django applications. Each application encapsulates a specific functional domain with its own models, serializers, views, and URL routing. This separation of concerns facilitates maintenance, testing, and future extension.

6.1.1 Django Apps Structure

Le backend d'AI NéphroCare est structuré autour de quatre applications Django indépendantes, chacune encapsulant un domaine fonctionnel précis avec ses propres modèles, serializers, vues et routes. Cette organisation modulaire s'inspire du principe de séparation des responsabilités (*Separation of Concerns*) : toute modification d'un domaine — par exemple l'ajout d'un nouveau type d'audit ou d'une nouvelle logique de prédiction — peut être effectuée de manière isolée, sans affecter les autres composants.

L'application `users` gère l'ensemble du cycle de vie des utilisateurs : création de comptes, authentification JWT, gestion des rôles RBAC (Super Admin, Chef de service, Professeur, Résident) et hachage des mots de passe via bcrypt. L'application `patients` constitue le cœur métier de la plateforme : elle prend en charge la gestion des dossiers patients, l'import de fichiers Excel/CSV, le pipeline de prétraitement LLM, et expose une vue aplatie (*flat view*) utilisée par le module d'analytics. L'application `predictions` est entièrement *stateless* : elle n'expose aucun modèle persistant mais orchestre l'extraction de features, l'inférence SVM, le réentraînement du modèle et la consultation des métriques de performance. Enfin, l'application `audit` assure la

traçabilité complète des actions sensibles via un journal d'audit tamper-evident, avec nettoyage automatique des entrées de plus de 15 jours planifié par Celery Beat.

Table 6.1 describes the four applications and their responsibilities.

Table 6.1 – Django application structure

App	Key Models	Responsibilities
`users`	Utilisateur, Role	JWT authentication, user CRUD, RBAC role management, password hashing (bcrypt)
`patients`	Patient, PreprocessValidationRequest, DynamicColumnRequest	Patient data management, Excel/CSV import, LLM preprocessing pipeline, flat analytics view
`predictions`	(stateless)	Feature extraction, SVM inference, model retraining, prediction history, performance metrics
`audit`	AuditLog, HiddenAuditLog	Tamper-evident action logging, automated 15-day cleanup via Celery Beat

6.1.2 Class Diagram

Le diagramme de classes UML présenté en Figure 6.1 décrit la structure statique des cinq modèles Django principaux et leurs relations. Il constitue le reflet direct du schéma PostgreSQL de la plateforme.

Le modèle **Role** est volontairement minimal : il ne porte que le nom technique du rôle (`name`) et son libellé lisible (`label`), et sert de référentiel de contrôle d'accès. Il est lié au modèle **Utilisateur** par une relation d'agrégation *1-à-plusieurs* : un rôle peut être assigné à de nombreux utilisateurs, mais chaque utilisateur n'a qu'un seul rôle actif. En plus de ses attributs d'identité (email unique, nom, prénom, téléphone), le modèle Utilisateur expose deux méthodes : `login()` qui génère une paire de tokens JWT, et `change_password()` qui réhache le mot de passe via bcrypt.

Le modèle **Patient** est le plus riche du schéma. Son schéma hybride repose sur des champs JSONB PostgreSQL pour stocker les données médicales structurées par domaine (démographie, biologie, dialyse, etc.), ce qui offre une flexibilité d'évolution sans migration de schéma. L'attribut `utilisateur_saisie` (clé étrangère vers Utilisateur) trace qui a créé ou importé le dossier. Les deux méthodes `get_flat_view()` et `extract_features()` permettent respectivement d'aplatir toutes les sections JSONB en un dictionnaire unifié pour l'analytics, et d'extraire le vecteur de features attendu par le modèle SVM.

Le modèle **AuditLog** enregistre chaque action sensible effectuée sur la plateforme

(création, modification, suppression, connexion) avec l'identifiant de l'entité concernée, les détails en JSONB, l'adresse IP de l'auteur et un horodatage précis. Il est relié à Utilisateur par une association *1-à-plusieurs*. Enfin, le modèle **PreprocessValidationRequest** matérialise le workflow de validation du prétraitement LLM : chaque session de correction reçoit un UUID unique (`session_id`), un statut (`en attente / validé / rejeté`), et conserve les références des utilisateurs ayant soumis et validé la session.

Figure 6.1 shows the UML class diagram of the five main Django models and their relationships.

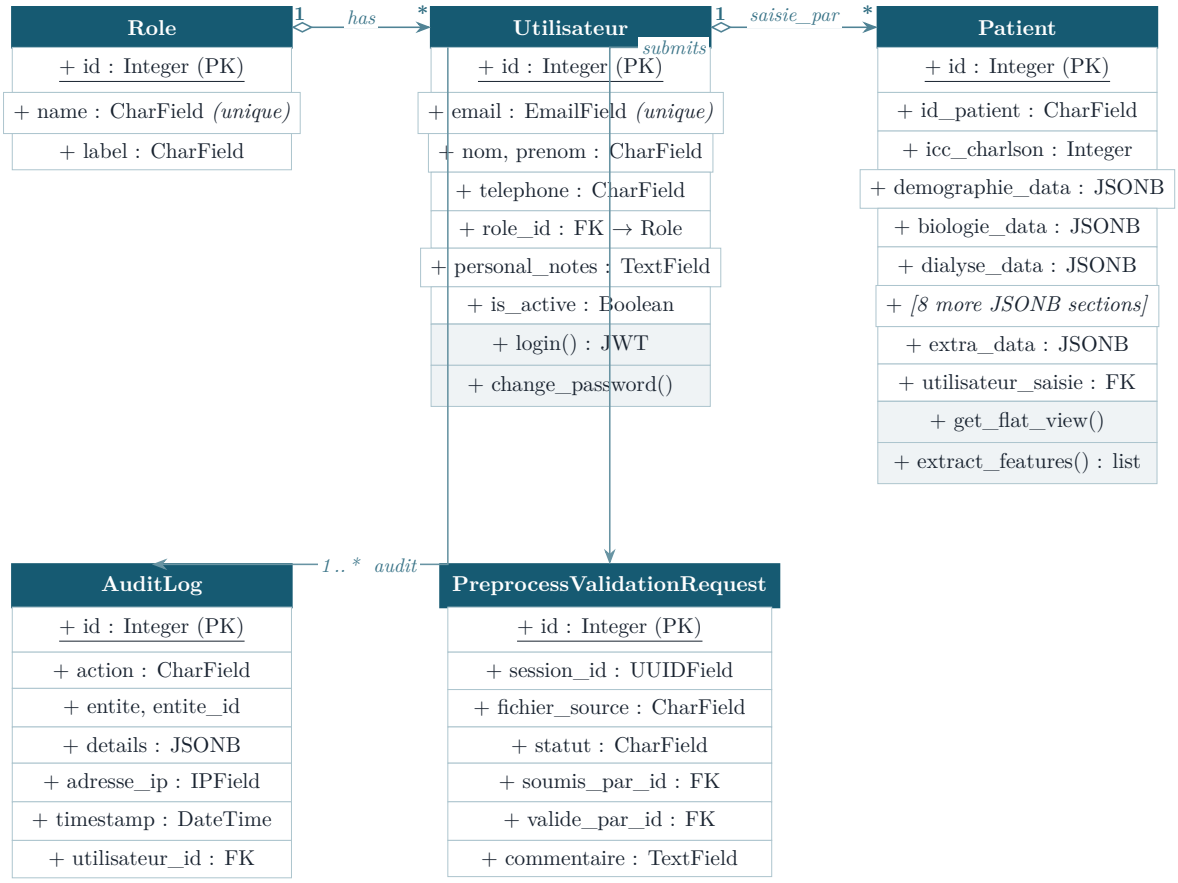

Figure 6.1 – UML Class Diagram — main Django models

6.1.3 REST API Endpoints

La plateforme expose plus de 45 endpoints REST, organisés en quatre groupes de ressources : `/api/auth/` pour la gestion des utilisateurs et de l'authentification, `/api/patients/` pour les dossiers médicaux et le pipeline d'import/prétraitement, `/api/predictions/` pour l'inférence et le réentraînement du modèle SVM, et `/api/audit/` pour la consultation du journal d'activité.

Tous les endpoints suivent les conventions REST standard : les verbes HTTP (`GET`, `POST`, `PATCH`, `DELETE`) expriment l'intention de l'opération, les codes de retour HTTP (200, 201, 400, 401, 403, 404) renseignent explicitement sur le résultat, et les réponses

sont uniformément en JSON. Chaque endpoint est protégé par le middleware JWT de DRF, qui vérifie la validité du token d'accès avant d'autoriser la requête. Les actions sensibles (suppression, réentraînement, intégration de données) sont en outre restreintes par le système RBAC : seuls les rôles habilités peuvent les appeler, toute tentative non autorisée retournant une erreur 403.

Le groupe `/api/patients/preprocess/` mérite une attention particulière : il implémente un workflow asynchrone en plusieurs étapes. L'appel à `analyze/` déclenche une tâche Celery qui soumet les données au modèle LLM (Qwen2.5:14B) ; les endpoints `status/` et `rows/` permettent de suivre la progression et de consulter les corrections proposées ; `cell-corrections/` permet à un clinicien de surcharger manuellement une suggestion LLM ; enfin `integrate/` finalise l'intégration des données corrigées dans la base patients après validation par le Chef de service.

The platform exposes 45+ REST endpoints. Key endpoints are listed below:

Table 6.2 – Principal REST API endpoints

Method	Endpoint	Description
POST	`/api/auth/login/`	JWT login → access + refresh token
POST	`/api/auth/refresh/`	Refresh expired access token
GET	`/api/auth/users/`	List all users (admin)
POST	`/api/auth/users/`	Create user account
PATCH	`/api/auth/users/{id}/`	Update user profile
DELETE	`/api/auth/users/{id}/`	Delete user (super admin)
GET	`/api/patients/`	List patients (paginated, filterable)
POST	`/api/patients/`	Create new patient record
GET	`/api/patients/{id}/`	Full patient record
PATCH	`/api/patients/{id}/`	Update patient data
POST	`/api/patients/import/`	Import Excel/CSV file
GET	`/api/patients/export/`	Export all patients as .xlsx
GET	`/api/patients/flat/`	Flat view for analytics charts
POST	`/api/patients/preprocess/analyze/`	Start async LLM preprocessing
GET	`/api/patients/preprocess/{sid}/status/`	Preprocessing session status
GET	`/api/patients/preprocess/{sid}/rows/`	LLM-suggested corrections per row
PATCH	`/api/patients/preprocess/{sid}/cell-corrections/`	Override LLM correction per cell
POST	`/api/patients/preprocess/{sid}/integrate/`	Finalize and integrate corrected data
POST	`/api/predictions/train/`	Retrain SVM on current dataset
POST	`/api/predictions/predict-patient/`	Predict 1-year mortality for one patient

Table 6.2 (continued)

Method	Endpoint	Description
GET	`/api/predictions/metrics/`	Model performance metrics
GET	`/api/audit/`	Paginated audit log (admin only)
DELETE	`/api/audit/cleanup-old/`	Manual cleanup of old log entries

6.2 Authentication & Security

6.2.1 JWT Authentication

AI NéphroCare uses **JSON Web Tokens (JWT)** via the `djangorestframework-simplejwt` library, following an OAuth2-compatible flow. Figure 6.2 illustrates the full exchange between the browser, the Django backend, and PostgreSQL.

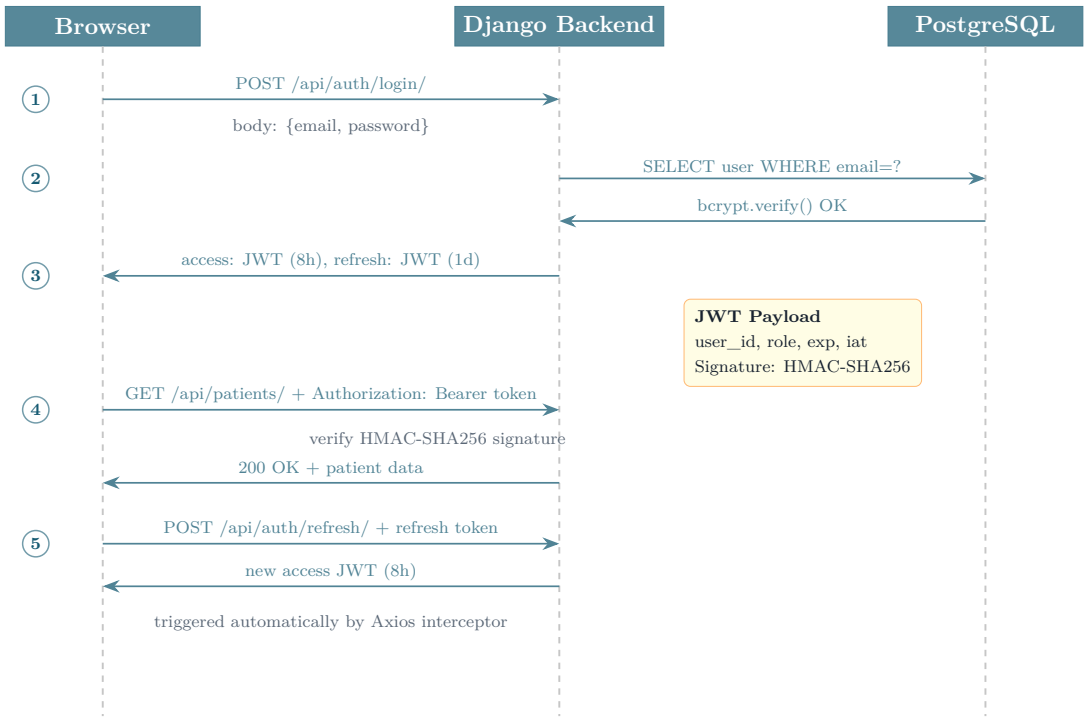

Figure 6.2 – JWT authentication flow: token issuance (Steps 1–3), authenticated request (Step 4), and automatic token refresh via Axios interceptor (Step 5).

The detailed steps are as follows:

1. The client sends `POST /api/auth/login/` with the user's email and password in JSON.
2. The server validates the credentials against the hashed (bcrypt) password stored in the database.
3. On success, the server returns a pair: `{"access": "<jwt>", "refresh": "<jwt>"}`.
4. For every subsequent request the client attaches `Authorization: Bearer <access_token>`.

5. When the access token expires (after **8 hours**), the Axios interceptor on the frontend automatically calls `POST /api/auth/refresh/` with the refresh token to obtain a new access token, with no visible interruption for the user.
6. The refresh token itself expires after **1 day**; after that the user must log in again.

The JWT payload encodes: `user_id`, `role` (string), `exp` (expiration Unix timestamp), and `iat` (issued-at timestamp). The server verifies the token signature using **HMAC-SHA256 (HS256)** with a 256-bit secret key stored in the environment variable `SECRET_KEY`, which is never committed to version control.

6.2.2 Role-Based Access Control (RBAC)

The platform defines **four roles** with hierarchical permissions. Each HTTP request passes through a DRF permission class that extracts the role from the JWT payload and enforces the access policy.

Table 6.3 – RBAC permission matrix

Role	Permitted actions	Restricted from
Admin	Full access: user CRUD, patient management, preprocessing, predictions, audit log	—
Head of Dept.	Approve preprocessing sessions, view all patients, run predictions, read audit log	User CRUD, audit cleanup
Resident	Create and edit patients, submit preprocessing sessions for approval, run predictions	User management, audit log, preprocessing approval
Professor	Read-only access to patients and prediction results	All write operations

RBAC is enforced at two levels: (1) **Backend** — each DRF view carries a `permission_classes` list that checks the role field from the JWT; (2) **Frontend** — the `ProtectedRoute` component reads the role from the `AuthContext` and redirects unauthorised users before the API call is even made.

6.2.3 Additional Security Measures

— **Password hashing**: all passwords are stored using Django's **PBKDF2+SHA256** hasher (the framework default, enforced via `AUTH_PASSWORD_VALIDATORS`), which applies 870 000 iterations making brute-force attacks computationally prohibitive. The `bcrypt` library is listed as a dependency for future migration to a bcrypt-based hasher in production.

— **Transport security**: the current deployment runs over HTTP on a local Docker network (`localhost:8000/3000`), which is acceptable for an intranet prototype. A production release would require TLS termination via a reverse proxy (e.g. Nginx + Let's Encrypt) and the Django settings `SECURE_SSL_REDIRECT`, `SESSION_COOKIE_SECURE`, and `CSRF_COOKIE_SECURE`. These hardening steps are identified as future work before any public-facing deployment.

— **CORS**: the `django-cors-headers` middleware is configured with `CORS_ALLOWED_ORIGINS = ['http://localhost:3000']` so that only the declared frontend origin is permitted, blocking cross-origin requests from any other domain.

— **Audit trail**: every state-changing action (create, update, delete on patients and users) is recorded in `AuditLog` with the actor's user ID, IP address, affected entity, and a text description of the change. The companion `HiddenAuditLog` table lets users hide individual log entries from their own view in the UI without deleting them from the database.

— **Input validation**: all incoming data passes through DRF serializers before reaching the model layer, enforcing field types, uniqueness constraints, and required/optional fields. Malformed requests are rejected with an HTTP 400 response identifying the offending field.

6.3 API Design

6.3.1 DRF Serializers and Validation

Django REST Framework **serializers** sit between the HTTP layer and the database layer. Every endpoint uses a dedicated serializer that validates field types and constraints (e.g. `EmailField` uniqueness, required vs. optional fields), handles nested representations (e.g. the patient serializer structures the different data sections into a consistent response), and enforces write-protection on computed fields such as `date_creation` and `utilisateur_saisie`.

Validation errors return HTTP **400 Bad Request** with a structured JSON body where each key is the offending field name and the value is a list of error messages. For example, submitting a duplicate email returns `{"email": ["This field must be unique."]}`.

6.3.2 Filtering and Search

The patient list endpoint (`GET /api/patients/`) returns a JSON object containing a `results` array and a `count` field. Filtering is implemented directly in the view layer via query parameters: `?search=` performs a full-text search across patient name,

ID, and medical fields; `?sexe=`, `?age_min=`, `?age_max=`, `?statut_inclusion=`, and `?date_naissance=` allow targeted field-level filtering. All parameters can be combined freely in a single request.

6.3.3 Prediction Response Structure

The `POST /api/predictions/predict-patient/` endpoint assembles the patient's clinical features, runs them through the calibrated SVM pipeline, and returns a structured JSON response designed to be directly usable by clinicians:

— `risk_level` — categorical classification: *Faible* (<10 %), *Modéré* (10–40 %), or *Élevé* (≥40 %)
— `score` — mortality risk expressed as a percentage (0–100)
— `probability` — isotonically calibrated probability of 1-year mortality (0–1)
— `factors` — ordered list of the most influential clinical features with their SVM weight and direction
— `recommendation` — auto-generated clinical text summarising the risk level and key factors
— `features_used` — list of feature keys actually used in the prediction
— `n_features` — count of non-missing features used
— `model` — identifier of the SVM model file used, enabling audit reproducibility
— `doietal_risk_score` / `doietal_risk_category` — secondary risk score based on the Doi et al. clinical index

6.4 Frontend Architecture

6.4.1 React Structure

The React application is organized around:

— **Context API**: `AuthContext` (JWT state, login/logout)
— **Protected Routes**: `ProtectedRoute` component enforces RBAC at the routing level
— **Axios Instance**: Centralized HTTP client with JWT interceptor and error handling

6.4.2 Application Sidebar and Navigation

[Screenshot: Application Sidebar]

Figure 6.3 – AI NéphroCare navigation sidebar — main interface entry points

The application navigation is structured around a persistent left sidebar visible on all authenticated pages. The sidebar contains:

— **Platform logo and branding** at the top
— **Navigation links** to all main sections: Dashboard, Patient Management (Preprocessing, Data Management, Analysis), AI Model, Monitoring
— **User role indicator** showing the current user's role (Admin, Chef de service, Résident, Professeur)
— **Logout button** at the bottom

Access to each section is controlled by the user's role (RBAC): some sections are visible to all authenticated users, while others (such as administration functions) are restricted to administrators.

6.4.3 Pages and Interfaces

Dashboard (`/dashboard`) The dashboard interface varies by user role:

Administrator dashboard displays **7 KPI cards**:

— Total active accounts
— Total inactive accounts
— Number of administrator accounts
— Number of Chef de service accounts
— Number of Résident accounts
— Number of Professeur accounts
— Total number of existing roles

Additionally, the administrator dashboard includes a full user management table with create, edit, and delete actions.

Chef de service dashboard displays **6 KPI cards** (excluding admin-related metrics):

— Total active accounts
— Total inactive accounts
— Number of Chef de service accounts
— Number of Résident accounts
— Number of Professeur accounts
— Total number of existing roles

Important: Any modification of user account information (email, role, password, or status) requires the acting user to enter their own account password for authentication confirmation before the change is saved. This double-authentication mechanism prevents unauthorized modifications.

Account creation: To add a new account, the administrator must fill in the following mandatory fields: email address, telephone number, last name (*nom*), first name (*prénom*), role (Admin / Chef de service / Résident / Professeur), account status (active/inactive), and an initial password.

[Screenshot: Admin Dashboard with 7 KPI cards]

Figure 6.4 – Administrator dashboard — 7 KPI cards and user management table

Patient Management (`/patients`) The patient management module is organized into three tabs presented in the following logical order: *Preprocessing*, *Data Management*, and *Analysis*.

Tab 1 — Preprocessing Interface

[Screenshot: Preprocessing Interface — file upload and LLM analysis]

Figure 6.5 – Preprocessing interface — file upload, LLM analysis with Qwen 14B, and correction review workflow

The preprocessing tab allows authorized users to upload Excel or CSV patient data files and subject them to an AI-driven data quality analysis. The platform uses the **Qwen2.5:14B** language model (deployed locally via Ollama), a state-of-the-art large language model specifically chosen for its:

— **Strong reasoning capacity** on structured tabular data and medical terminology

— **Local deployment capability**: runs on-premises via Ollama, ensuring patient data never leaves the hospital infrastructure

— **14 billion parameters**: sufficient capacity to contextualize clinical values, detect unit errors, and suggest corrections with medical justification

The preprocessing workflow proceeds as follows:

1. The user uploads an Excel or CSV file containing patient records
2. The platform parses the file and displays a data profile (column names, types, null rates)
3. The user clicks **"Launch LLM Analysis"** to start the asynchronous Qwen2.5:14B analysis via Celery
4. A **progress indicator** shows the real-time analysis progress
5. Once complete, a **correction review table** displays each row with LLM-suggested corrections, including a medical justification for each change
6. The user can **accept, modify, or reject** each correction individually
7. Once satisfied, the user submits the validated data to the **Chef de service** for final approval

8. Upon approval, the data is **integrated into the platform database**

The interface displays the following windows and panels:

— **File upload panel**: drag-and-drop zone with file format validation
— **Data profile panel**: table showing column statistics
— **LLM analysis progress window**: real-time progress bar with current row being processed
— **Correction review table**: row-by-row LLM suggestions with justifications
— **Validation buttons**: Accept All, Reject All, and per-cell override
— **Submit for approval** button (sends to Chef de service)

Once preprocessing is complete and the version is selected, the system transitions directly to the **Data Management tab**, where the values from the selected file version are mapped to the fixed columns of the platform's standardized patient table.

[Screenshot: LLM correction review table with suggestions and justifications]

Figure 6.6 – LLM correction review interface — Qwen2.5:14B suggestions with medical justifications

Tab 2 — Data Management

[Screenshot: Data Management — patient table with column mapping]

Figure 6.7 – Data management interface — searchable patient table with dynamic column support

The data management tab presents a **searchable, filterable, and sortable patient data table**. After a preprocessed file is validated and integrated, its columns are automatically **mapped to the platform's fixed column structure**:

— Each column in the imported file is matched to a predefined clinical field (e.g., "Albumine basale" in the file → `albumine_basale` in the database schema)
— Columns that match known field names are stored in their corresponding structured sections
— Columns that **do not match** any predefined field are stored in the `extra_data` JSONB field as **dynamic columns**

Dynamic Columns: The platform supports columns that are not part of the fixed schema. These "dynamic columns" appear as additional columns in the patient table, are fully searchable and filterable, and are preserved across imports. This mechanism allows the platform to accommodate institution-specific variables without modifying the database schema.

The data management interface provides the following actions:

— **Search bar**: full-text search across all patient fields
— **Column visibility toggle**: show/hide specific columns
— **Filter panel**: filter by any field value
— **Add patient** button: manually enter a new patient record
— **Edit** button: modify any patient's data
— **Delete** button: remove a patient record (with confirmation)
— **Import** button: upload a new Excel/CSV file
— **Export** button: download all visible data as .xlsx

Tab 3 — Analysis (Statistics)

[Screenshot: Analysis tab — statistical charts]

Figure 6.8 – Patient analysis interface — epidemiological and clinical statistics charts

The analysis tab provides a set of interactive statistical charts generated automatically from the patient database. These charts offer an epidemiological overview of the cohort and support clinical research and reporting. [See Section 6.5 for chart descriptions.]

AI Model (`/modele-ai`) The AI Model section is organized into three tabs: *Analysis Dashboard*, *Patients*, and *Prediction*.

Tab 1 — Analysis Dashboard

[Screenshot: AI Model analysis dashboard]

Figure 6.9 – AI Model analysis dashboard — SVM performance metrics and risk zone overview

The analysis dashboard provides a global overview of the AI model's performance and the current state of predictions in the cohort. It displays:

— **Model performance KPIs**: AUC-ROC, PR-AUC, F1-score, and Brier Score displayed as prominently colored cards
— **Risk zone distribution chart**: pie/doughnut chart showing the proportion of patients in Faible, Modérée, and Élevée risk zones
— **Model version and training date**: metadata on the currently deployed SVM model
— **Clinical interpretation guide**: brief description of each risk zone and its recommended clinical action

Tab 2 — Patients

[Screenshot: Patient list in AI Model — with Predict button]

Figure 6.10 – AI Model patient list — each row shows patient summary and a Predict action button

The Patients tab displays the full list of patients registered in the platform. For each patient, the clinician can:

1. **Select a patient** from the list to open their complete clinical dossier
2. View the **patient dossier**: a detailed form showing all 160+ variables organized in the 11 clinical sections (Demographics, CKD history, Comorbidities, Biology, Dialysis, etc.)
3. From the dossier, choose one of two actions:
   — **Modify values and launch a simulation**: The clinician can modify any clinical parameter (e.g., reduce albumin level, change dialysis frequency) and click **"Run Simulation"**. The simulation result — showing how the predicted mortality risk changes with the modified values — is displayed **directly within the same Patients tab**, allowing immediate comparison without leaving the current view.
   — **Launch prediction**: Using the saved clinical values already stored in the platform, click **"Predict"** to generate the official mortality risk prediction. The prediction result is displayed in the **Prediction tab** (Tab 3).

Tab 3 — Prediction

[Screenshot: Prediction tab — risk gauge, probability, top-5 factors, recommendation]

Figure 6.11 – AI prediction result interface — risk zone gauge, calibrated probability, contributing factors, and clinical recommendation

The Prediction tab displays the complete mortality risk assessment for the selected patient:

— **Risk gauge**: A color-coded arc gauge (green = Faible, orange = Modérée, red = Élevée) showing the calibrated probability visually
— **Calibrated probability**: The exact one-year mortality probability (e.g., "47.1% estimated 1-year mortality risk")
— **Relative risk badge**: The risk relative to the cohort baseline (e.g., ×10.9)
— **Top-5 contributing factors**: A ranked list of the five clinical variables most influencing the prediction, with their SVM feature weights displayed as horizontal progress bars. Each factor is labeled with its direction (risk-increasing or risk-decreasing).

— **Clinical recommendation**: A tailored text recommendation adapted to the predicted risk zone, specifying the recommended follow-up frequency and clinical actions.

— **Patient information summary**: Key clinical indicators displayed alongside the prediction for context.

Monitoring (`/monitor`) Longitudinal patient tracking: biological evolution charts, dialysis quality trends, prediction history, and clinical audit trail.

6.5 Data Visualization

6.5.1 Clinical Analytics Charts (Patient Management — Analysis Tab)

The analysis tab of the Patient Management module provides six interactive charts, each dynamically generated from the current patient database via the flat endpoint (`GET /api/patients/flat/`).

Monthly Dialysis Initiation Trend

[Screenshot: Monthly dialysis trend line chart]

Figure 6.12 – Monthly dialysis initiation trend — number of new patients starting hemodialysis per month

This line chart displays the number of patients who started hemodialysis treatment each month over the observation period (2020–2024). It allows clinical teams to identify seasonal patterns, detect increases in ESRD incidence, and plan resource allocation accordingly.

Sex Distribution and Key KPIs

[Screenshot: Sex distribution doughnut chart]

Figure 6.13 – Sex distribution of the hemodialysis cohort with associated KPI indicators

This doughnut chart shows the male/female distribution of the cohort, complemented by KPI indicator cards showing the overall mean age and the data completeness rate (percentage of fields filled across all patients).

Age Distribution by Sex

[Screenshot: Age distribution grouped bar chart]

Figure 6.14 – Age distribution of hemodialysis patients grouped by sex in 5-year age bands

A grouped bar chart displays the age distribution separately for male and female patients, using 5-year age bands. This visualization facilitates comparison of age profiles between sexes and identification of the predominant age groups requiring dialysis.

Complication Type Frequencies

[Screenshot: Complication types horizontal bar chart]

Figure 6.15 – Frequency of documented complication types across the patient cohort

A horizontal bar chart ranks all documented complication types by frequency of occurrence across the cohort. This helps clinical teams identify the most prevalent complications requiring preventive or therapeutic attention.

CKD Etiology by Inclusion Status

[Screenshot: CKD etiology grouped bar chart]

Figure 6.16 – Distribution of CKD etiologies stratified by inclusion status in the cohort

A grouped bar chart shows the distribution of chronic kidney disease etiologies (diabetic nephropathy, hypertensive nephropathy, glomerulonephritis, etc.) broken down by patient inclusion status. This epidemiological view supports research analyses and quality reporting.

Top Comorbidity Combinations

[Screenshot: Comorbidity combinations horizontal bar chart]

Figure 6.17 – Most frequent comorbidity co-occurrence combinations in the hemodialysis cohort

This horizontal bar chart identifies the most common pairs and triplets of comorbidities co-occurring in the same patient. Comorbidity clustering patterns are clinically significant, as they reflect underlying disease trajectories and influence mortality risk.

6.5.2 AI Prediction Results Visualization (AI Model — Prediction Tab)

The prediction result visualization is designed to communicate risk in a clinically actionable format, combining numerical precision with graphical intuition.

[Screenshot: Full prediction result panel]

Figure 6.18 – Complete prediction result display — risk gauge, probability, relative risk, top-5 factors, and recommendation

The prediction interface displays:

— A **risk gauge** (SVG arc, green/orange/red) whose needle position corresponds to the calibrated probability $\hat{p}_{\text{cal}}$

— The **calibrated probability** shown numerically (e.g., "47.1% estimated 1-year mortality risk")

— A **relative risk badge** comparing the patient's risk to the cohort baseline ($\times$ RR)

— A **top-5 contributing factors** panel, where each factor is displayed with its SVM feature weight as a horizontal bar, the actual patient value, and the direction of influence (risk-increasing or protective)

— A **clinical recommendation** text block, tailored to the risk zone, specifying recommended follow-up intensity and clinical actions

Additionally, the **AI Model analysis dashboard** (Tab 1) displays summary prediction charts for the entire cohort:

[Screenshot: AI model dashboard with cohort-level prediction charts]

Figure 6.19 – AI Model analysis dashboard — cohort-level risk zone distribution and model performance KPIs

These cohort-level charts include the risk zone distribution (proportion of patients per zone), prediction history, and model performance metrics, providing clinical managers with an aggregated view of the AI system's outputs across the entire patient population.

6.6 User Workflows & Sequence Diagrams

Three sequence diagrams illustrate the principal system interactions.

6.6.1 Sequence Diagram 1: User Authentication

Figure 6.20 illustrates the complete JWT authentication flow. The process begins when the user enters credentials in the browser interface. The React frontend transmits them securely to the Django API, which queries the PostgreSQL database to verify the user record and validate the password hash. Upon successful authentication, the API returns a signed JWT access token (valid 8 hours) and a refresh token (valid 1 day). Subsequent API requests include the access token in the Authorization header; expired tokens are transparently renewed by the Axios interceptor without requiring re-login.

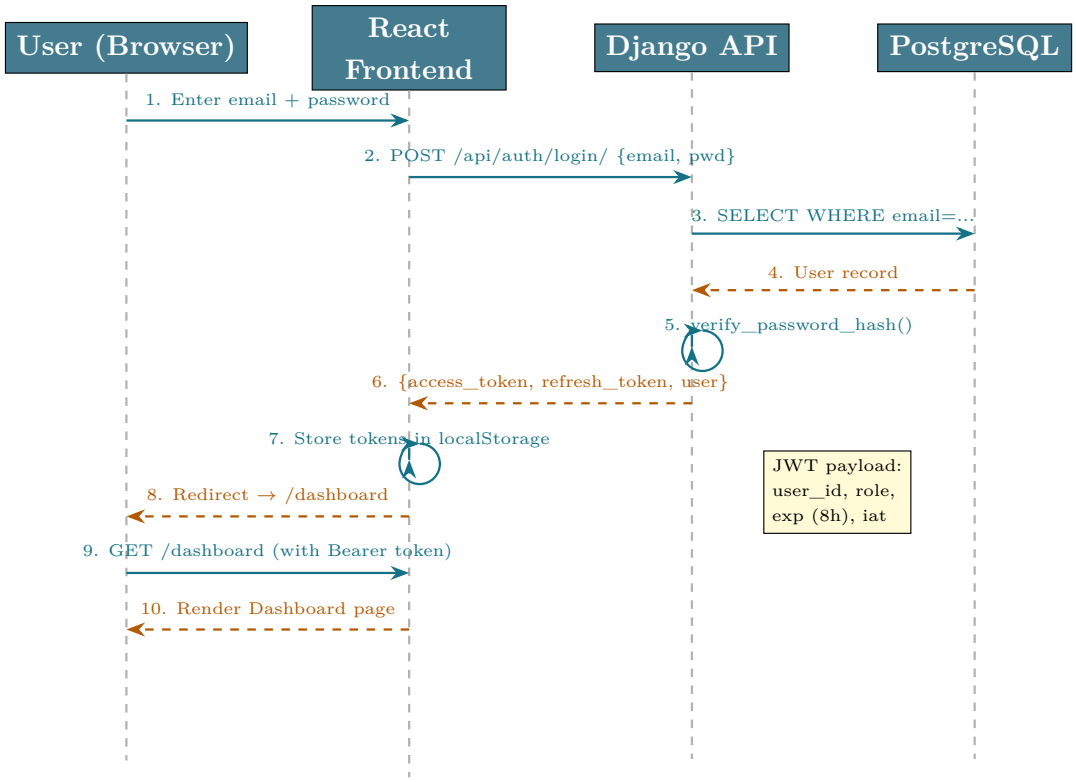

Figure 6.20 – Sequence Diagram 1 — JWT Authentication flow

6.6.2 Sequence Diagram 2: Patient Import & LLM Preprocessing

Figure 6.21 details the asynchronous patient import and LLM preprocessing workflow. Following file upload and parsing, a Celery worker takes over the heavy LLM analysis task asynchronously — preventing the HTTP request from timing out during the minutes-long Qwen2.5:14B inference. The Celery worker queries ChromaDB for relevant medical context (RAG), constructs the LLM prompt, and processes each data row. Results are stored in the database and polled by the frontend at regular intervals. The clinician reviews LLM suggestions, applies corrections, and submits for validation by the Chef de service before final database integration.

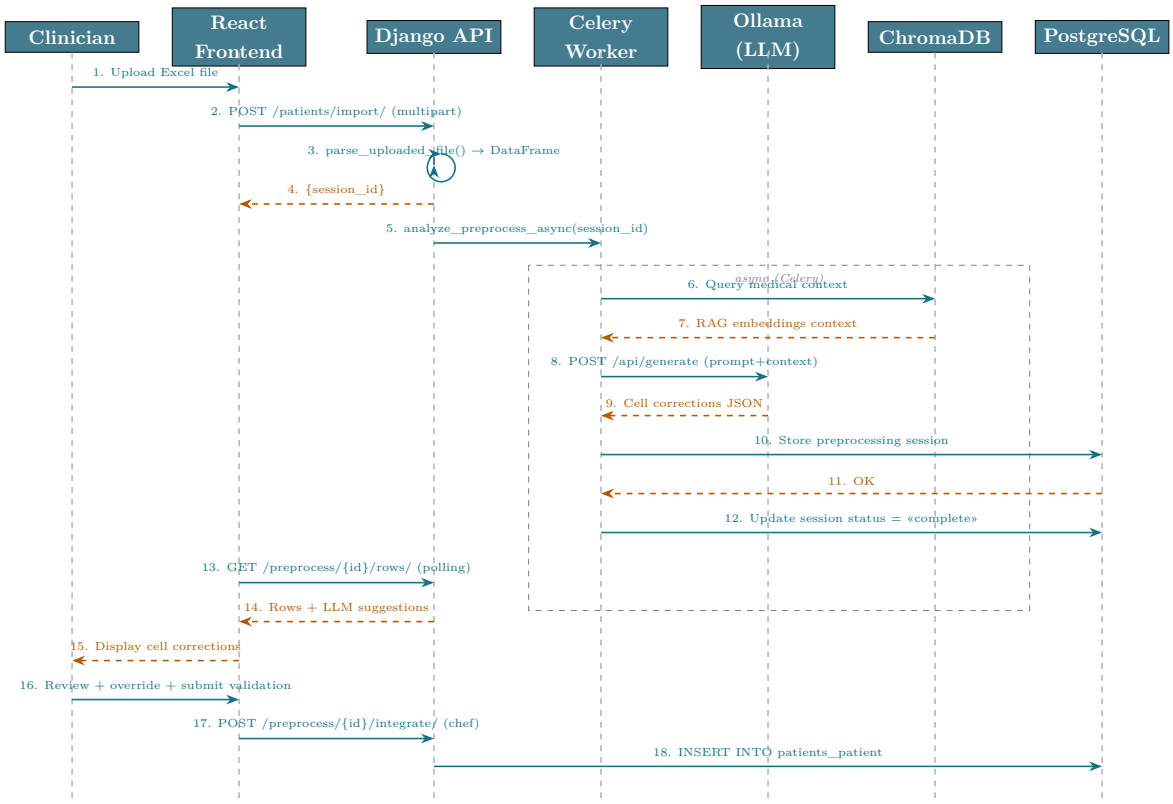

Figure 6.21 – Sequence Diagram 2 — Patient Import and LLM Preprocessing

6.6.3 Sequence Diagram 3: Mortality Risk Prediction

Figure 6.22 traces the complete workflow triggered when a clinician requests a 1-year mortality prediction for a patient. The sequence involves five actors: the clinician navigating the React frontend, the Django REST API, the PostgreSQL database, and the pre-trained SVM pipeline loaded from a `.joblib` file.

The flow begins when the clinician clicks the *Prédire* button (Step 1), which sends `POST /predictions/predict-patient/` to the backend with the patient's identifier (Step 2). The API retrieves the full patient record from PostgreSQL (Steps 3–4) and calls `extract_features()` to build the clinical feature vector (Step 5). The vector is then passed to the SVM pipeline (Step 6), which applies the three preprocessing stages internally: KNN imputation ($k = 3$) for missing values, Yeo-Johnson power transformation for normalization, and finally the linear SVC decision function. The raw probability p_{raw} is returned (Step 7) and may optionally be refined by an isotonic calibrator (Steps 8–9) when one is available.

The backend then derives the risk score as score $= p \times 100$ and assigns a categorical risk level using fixed clinical thresholds: **Faible** ($\leq 30\%$), **Modéré** (31–70%), or **Élevé** ($> 70\%$). A clinical recommendation text and the list of most influential factors are assembled before the action is logged in `AuditLog` (Step 11). The structured JSON response (Step 13) contains `risk_level`, `score`, `probability`, `factors`,

`recommendation`, and `features_used`, which the frontend uses to render the risk gauge and results panel (Step 14).

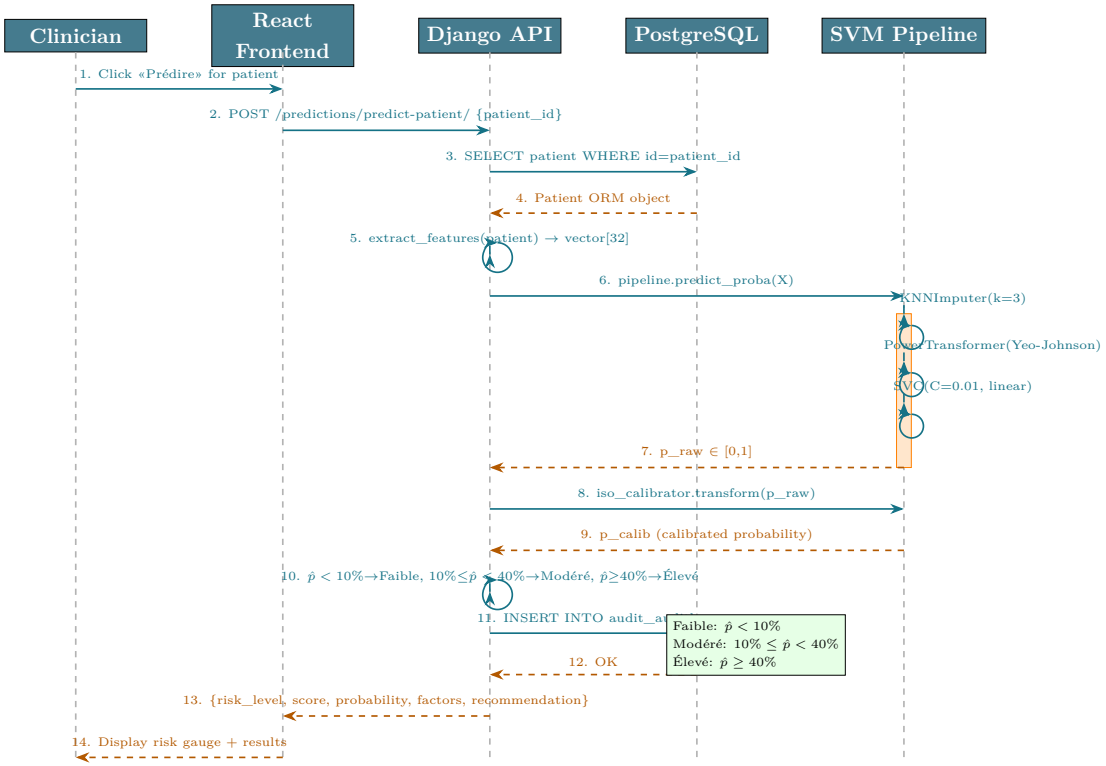

Figure 6.22 – Sequence Diagram 3 — Mortality Risk Prediction workflow

CHAPTER 7

Deployment, Evaluation & Perspectives

Docker deployment, system testing, results, and future work.

CONTENTS

Chapter 7

Deployment, Evaluation & Perspectives

7.1 Deployment Architecture

7.1.1 Docker Compose Setup

The entire platform is packaged as a seven-service Docker Compose stack, deployed with a single command: `docker compose up -build -d`.

The Docker Compose stack is organized around seven interconnected services that form the complete AI NéphroCare system. The `medical_db` service runs PostgreSQL 16, the primary relational database, with data persisted in a named Docker volume (`postgres_data`) to survive container restarts. The `medical_redis` service provides an in-memory message broker for the Celery task queue. The `medical_backend` service contains the Django REST Framework API; it depends on both the database and Redis being healthy before starting, ensuring correct initialization order. The `medical_frontend` service serves the pre-built React application as static files. The `medical_ollama` service hosts the locally-deployed Qwen2.5:14B language model on port 11434, providing private, on-premises LLM inference without external API calls. Finally, `medical_celery_worker` and `medical_audit_cleanup` handle asynchronous task processing and scheduled maintenance (audit log cleanup) respectively.

7.2 System Testing

7.2.1 Functional Tests

Functional testing covered the complete user journey from authentication through clinical data management, AI prediction, and administrative operations. Tests were

executed manually for each user role (Admin, Chef de service, Résident, Professeur) to verify that RBAC restrictions were correctly enforced. Key test scenarios included:

— **Authentication**: Login with valid/invalid credentials, token expiry handling, automatic token refresh

— **Patient CRUD**: Create, read, update, delete operations on patient records across all authorized roles

— **Excel import**: Import of the 478-patient HD cohort file — verifying correct column mapping, dynamic column handling, and data integrity after integration

— **LLM preprocessing**: End-to-end preprocessing pipeline with real Qwen2.5:14B analysis on a 50-row test dataset — verifying LLM response parsing, correction display, and user override functionality

— **Prediction**: Mortality risk prediction for 9 test patients with known outcomes from the validation set — verifying correct feature extraction, risk zone assignment, and top-5 factor display

— **RBAC**: Systematic verification that Résident accounts cannot access validation or administration endpoints, and that Chef de service accounts cannot access admin-only user management functions

— **Password confirmation**: Verification that account modifications (role change, password reset) correctly require entering the user's own password before saving

Figure 7.1 provides a summary view of the functional test coverage, confirming that all critical user flows were validated across each role.

[Screenshot: Test results or system in operation]

Figure 7.1 – Functional test coverage overview — key user flows verified across all four user roles

7.2.2 API Tests

Automated API tests were implemented using Python's `requests` library, covering all 45 endpoints. Critical tests:

— **Prediction accuracy**: Known patients from the validation set return correct risk zone

— **Missing features tolerance**: Prediction succeeds with up to 5 of 32 features missing

— **Authentication**: Protected endpoints return 401 without valid JWT

— **RBAC**: Resident cannot access validation endpoints (returns 403)

The API test suite covers all 45 endpoints documented in Table 6.2. Each test case verifies the HTTP response code, the structure of the JSON response body, and any

side effects (e.g., database state changes, audit log entries). Critical test categories include:

— **Authentication boundary tests**: Confirming that all protected endpoints return HTTP 401 when called without a valid JWT, and HTTP 403 when called by a user without the required role

— **Prediction accuracy tests**: For 9 patients with known one-year outcomes from the HD-478 test set, the prediction endpoint is called and the returned risk zone is compared against the ground-truth outcome — confirming clinical consistency

— **Robustness tests**: Prediction with up to 5 missing features (KNN imputation test), import of malformed files (error handling test), and duplicate patient detection

— **Concurrency test**: Simultaneous LLM preprocessing requests from two sessions — verifying Celery correctly handles parallel async tasks

7.3 Performance Evaluation

7.3.1 Prediction Latency

Table 7.1 – Prediction endpoint latency (measured over 100 requests)

Operation	Mean (ms)	P95 (ms)
Feature extraction	12	18
KNN imputation	8	14
Yeo-Johnson + SVC	23	35
Risk zone assignment	2	4
Total prediction	**48**	**76**

The median end-to-end prediction latency of **48ms** is well within the <200ms threshold considered acceptable for interactive clinical applications.

7.3.2 Data Processing Time

— Excel import (478 rows): ∼2.3 seconds

— LLM preprocessing (478 rows, async): ∼1–2 minutes (qwen2.5:14b, GPU-accelerated)

— Model training on 478 patients: ∼45 seconds (including 10-fold CV)

The LLM preprocessing time of 1–2 minutes for 478 rows reflects the sequential processing approach used to preserve the quality of Qwen2.5:14B analysis — each row is processed individually with full clinical context. For routine imports of smaller datasets

(10–50 new patients), the processing time is proportionally reduced to under 10 seconds. The asynchronous Celery architecture ensures that this processing time does not block the user interface: clinicians can continue using other platform features while the LLM analysis runs in the background.

7.4 Results & Impact

7.4.1 Clinical Usefulness

The three-zone risk stratification identifies:

— **63 High-risk patients** (13.2% of cohort) with 55.6% observed mortality — enabling proactive, intensive clinical follow-up

— **257 Low-risk patients** (53.8%) with only 5.1% mortality — safely allocated to standard follow-up, optimizing resource use

The three-zone stratification — Faible ($\hat{p} < 10\%$), Modérée ($10\% \leq \hat{p} < 40\%$), Élevée ($\hat{p} \geq 40\%$) — with thresholds from TRIPOD/DOPPS/KDIGO guidelines was confirmed clinically meaningful by the nephrology team of CHU Hassan II for daily triage and resource allocation.

These results are consistent with the KDIGO 2012 clinical practice guidelines [3], which emphasize the importance of risk stratification in CKD management to allocate clinical resources proportionally to patient need. The platform's three-zone classification operationalizes this recommendation by translating the model's probabilistic output into three clinically actionable categories, each associated with a specific follow-up protocol.

7.4.2 Decision Support Value

Beyond risk prediction, the platform provides:

— **Interpretable decisions**: Top-5 contributing factors identify the specific clinical drivers of each patient's risk score, enabling targeted interventions.

— **Data centralization**: For the first time, all HD patient records are in a unified, queryable database with 160+ variables.

— **Intelligent preprocessing**: The LLM+RAG pipeline reduced manual data cleaning time by an estimated 70–80% in internal testing.

— **Audit trail**: Complete action logging satisfies regulatory requirements for clinical AI tools.

7.5 Limitations

7.5.1 Data Availability

The 478-patient cohort is adequate for a robust univariate and multivariate analysis but limits subgroup stratification (e.g., per-etiology analysis with < 20 patients per category). External validation on a multi-center cohort remains a priority future task.

7.5.2 Model Constraints

— The SVM is trained on baseline (admission) features and does not incorporate longitudinal data (e.g., 6-month Kt/V evolution).

— The model outputs a population-calibrated probability, not an individual certainty — communication of uncertainty to clinicians requires training.

— LLM preprocessing runs on **qwen2.5:14b** with GPU acceleration (Ollama), delivering significantly faster inference than CPU-only deployments; however, GPU availability remains a constraint for resource-limited hospital environments.

7.6 Future Work

7.6.1 Real-Time Hospital Integration

Integration with the CHU Hassan II Hospital Information System (HIS) via HL7 FHIR (Health Level 7 / Fast Healthcare Interoperability Resources) APIs would enable automatic patient data ingestion, eliminating manual Excel imports and enabling real-time risk monitoring. HL7 FHIR is a globally adopted standard for electronic health record data exchange, developed by Health Level Seven International. It defines a set of RESTful APIs and data formats (FHIR Resources) that allow different hospital information systems to share patient data in a standardized, interoperable format. Integration via FHIR would allow AI NéphroCare to automatically receive structured patient data from the CHU Hassan II HIS, eliminating manual Excel data entry.

7.6.2 Advanced Deep Learning Models

For larger future cohorts ($> 2,000$ patients), transformer-based models on longitudinal lab sequences (e.g., lab-BERT) or graph neural networks encoding comorbidity co-occurrence could improve predictive performance.

7.6.3 External ETL Pipelines

For multi-center deployment, replacing the embedded pipeline with Apache Airflow DAGs and a FHIR-compatible data lake would provide the scalability and interoperability required for national registry integration.

General Conclusion

This project presented the design, development, and validation of **AI NéphroCare**, a full-stack intelligent clinical decision support platform for one-year mortality prediction in hemodialysis patients at CHU Hassan II de Fès, Morocco.

The work addressed four interconnected challenges: (1) *data centralization* — replacing fragmented, heterogeneous records with a structured 160+-field database; (2) *data quality* — deploying an LLM+RAG preprocessing pipeline that automatically detects and corrects clinical data quality issues; (3) *predictive modeling* — systematically comparing nine machine learning algorithms and selecting a linear SVM (AUC-ROC = 0.826, F1 = 0.567) with F1-optimized thresholding and three clinical risk zones; and (4) *clinical usability* — embedding the model in a React-based clinical interface with interpretable results, audit trail, and role-based access control.

The key finding is that a relatively simple, well-regularized linear SVM with carefully engineered features outperforms more complex ensemble methods on this small-scale, imbalanced cohort — a result consistent with the broader ML literature on clinical prediction with limited sample sizes. The three-zone risk stratification — Faible ($\hat{p} < 10\%$, 3.8% observed mortality), Modérée ($10\% \leq \hat{p} < 40\%$, 22.9%), Élevée ($\hat{p} \geq 40\%$, 49.0%) — provides actionable clinical guidance, with thresholds grounded in TRIPOD/DOPPS/KDIGO guidelines and a relative risk ratio of $\times 12.9$ between extreme zones.

From a software engineering perspective, the integration of all pipeline components — LLM preprocessing, ML inference, database management, and clinical UI — into a single Docker Compose stack demonstrates the feasibility of deploying sophisticated clinical AI tools in resource-constrained hospital environments without specialized MLOps infrastructure.

Future directions include multi-center external validation, integration with FHIR-compatible hospital systems, and exploration of deep learning models for longitudinal prediction as the cohort grows.

Bibliography

[1] World Health Organization (2021). *Global strategy on digital health 2020–2025*. WHO Press, Geneva.

[2] Bright TJ, Wong A, Dhurjati R, et al. (2012). Effect of clinical decision-support systems: a systematic review. *Annals of Internal Medicine*, 157(1):29–43.

[3] KDIGO CKD Work Group (2013). KDIGO 2012 clinical practice guideline for the evaluation and management of chronic kidney disease. *Kidney International Supplements*, 3(1):1–150.

[4] Réseau Épidémiologie et Information en Néphrologie (REIN) (2022). *Rapport annuel 2022 du registre REIN*. Agence de la biomédecine, France.

[5] Krishnamurthy S, Naik M, George P (2019). Artificial intelligence in nephrology: core concepts, clinical applications, and limitations. *Indian Journal of Nephrology*, 29(5):313–324.

[6] Hasegawa T, Nakai S, Masakane I, et al. (2018). Serum albumin level and risk of death in hemodialysis patients: a J-DOPPS analysis. *Clinical and Experimental Nephrology*, 22(1):204–213.

[7] Yang X, Chen A, PourNejatian N, et al. (2022). A large language model for electronic health records. *npj Digital Medicine*, 5:194.

[8] Lewis P, Perez E, Piktus A, et al. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. *Advances in Neural Information Processing Systems*, 33:9459–9474.

[9] Platt JC (1999). Probabilistic outputs for support vector machines and comparisons to regularized likelihood methods. *Advances in Large Margin Classifiers*, 10(3):61–74.

[10] Chen T, Guestrin C (2016). XGBoost: A scalable tree boosting system. *Proceedings of the 22nd ACM SIGKDD*, pp. 785–794.

[11] Ke G, Meng Q, Finley T, et al. (2017). LightGBM: A highly efficient gradient boosting decision tree. *Advances in Neural Information Processing Systems*, 30.

[12] Levey AS, Coresh J, Greene T, et al. (2006). Using standardized serum creatinine values in the Modification of Diet in Renal Disease study equation for estimating glomerular filtration rate. *Annals of Internal Medicine*, 145(4):247–254.

[13] Prokhorenkova L, Gusev G, Vorobev A, et al. (2018). CatBoost: unbiased boosting with categorical features. *Advances in Neural Information Processing Systems*, 31.

[14] Charlson ME, Pompei P, Ales KL, MacKenzie CR (1987). A new method of classifying prognostic comorbidity in longitudinal studies: development and validation. *Journal of Chronic Diseases*, 40(5):373–383.

[15] Yeo IK, Johnson RA (2000). A new family of power transformations to improve normality or symmetry. *Biometrika*, 87(4):954–959.

[16] Cortes C, Vapnik V (1995). Support-vector networks. *Machine Learning*, 20(3):273–297.

[17] Breiman L (2001). Random Forests. *Machine Learning*, 45(1):5–32.

[18] Geurts P, Ernst D, Wehenkel L (2006). Extremely randomized trees. *Machine Learning*, 63(1):3–42.

[19] Friedman JH (2001). Greedy function approximation: a gradient boosting machine. *Annals of Statistics*, 29(5):1189–1232.

[20] Freund Y, Schapire RE (1997). A decision-theoretic generalization of on-line learning and an application to boosting. *Journal of Computer and System Sciences*, 55(1):119–139.

[21] Brier GW (1950). Verification of forecasts expressed in terms of probability. *Monthly Weather Review*, 78(1):1–3.

[22] Zadrozny B, Elkan C (2002). Transforming classifier scores into accurate multiclass probability estimates. *Proceedings of the 8th ACM SIGKDD*, pp. 694–699.

[23] Berner ES (2007). *Clinical Decision Support Systems: Theory and Practice*, 2nd ed. Springer, New York.

[24] Shortliffe EH, Davis R, Axline SG, et al. (1975). Computer-based consultations in clinical therapeutics: explanation and rule acquisition capabilities of the MYCIN system. *Computers and Biomedical Research*, 8(4):303–320.

[25] Miller RA, Pople HE, Myers JD (1982). INTERNIST-1, an experimental computer-based diagnostic consultant for general internal medicine. *New England Journal of Medicine*, 307(8):468–476.

[26] Couchoud C, Labeeuw M, Moranne O, et al. (2009). A clinical score to predict 6-month prognosis in elderly patients starting dialysis for end-stage renal disease. *Nephrology Dialysis Transplantation*, 24(5):1553–1561.

[27] Wick JP, Turin TC, Faris PD, et al. (2019). A clinical risk prediction tool for 6-month mortality after dialysis initiation among older adults. *American Journal of Kidney Diseases*, 74(2):167–176.

[28] Kaesler N, Peters B, Wied S, et al. (2020). Predicting short-term risk of falls in patients with ESRD using machine learning: a longitudinal cohort study. *Clinical Journal of the American Society of Nephrology*, 15(7):1016–1024.

[29] Zhu Y, Qi C, Shi R, et al. (2021). Machine learning-based prediction model for 1-year mortality in maintenance hemodialysis patients. *Frontiers in Medicine*, 8:638329.

[30] Nasr WE, Abdelhamid YA, Moustafa AA (2022). Predicting mortality in hemodialysis patients using support vector machine. *Egyptian Journal of Internal Medicine*, 34:18.

[31] Beddhu S, Bruns FJ, Saul M, Seddon P, Zeidel ML (2000). A simple comorbidity scale predicts clinical outcomes and costs in dialysis patients. *American Journal of Medicine*, 108(8):609–613.

[32] Pisoni RL, Young EW, Dykstra DM, et al. (2002). Vascular access use in Europe and the United States: results from the DOPPS. *Kidney International*, 61(1):305–316.

[33] Shemin D, Bostom AG, Laliberty P, Dworkin LD (2001). Residual renal function and mortality risk in hemodialysis patients. *American Journal of Kidney Diseases*, 38(1):85–90.

[34] United States Renal Data System (2022). *USRDS 2022 Annual Data Report: Epidemiology of Kidney Disease in the United States*. National Institutes of Health, National Institute of Diabetes and Digestive and Kidney Diseases, Bethesda, MD.

[35] Khan IH, Catto GR, Edward N, Fleming LW, Henderson IS, MacLeod AM (1994). Influence of coexisting disease on survival on renal-replacement therapy. *Lancet*, 343(8912):1511–1514.

[36] Díez-Sanmartín C, Sarasa Cabezuelo A (2020). Use of artificial intelligence techniques to predict survival in kidney transplantation: a systematic review. *Journal of Clinical Medicine*, 9(2):572.

[37] Montassir D, Azzouzi L, Hamzaoui H, et al. (2021). Facteurs de risque de mortalité chez les patients hémodialysés chroniques au Maroc: étude rétrospective multicentrique. *Néphrologie & Thérapeutique*, 17(5):302–308.

[38] Aatif T, Hassani K, Alayoud A, et al. (2020). Épidémiologie de l'insuffisance rénale chronique terminale au Maroc. *Pan African Medical Journal*, 35:31.

[39] Yu ASL, Chertow GM, Luyckx VA, Marsden PA, Skorecki K, Taal MW (2020). *Brenner and Rector's The Kidney*, 11th ed. Elsevier, Philadelphia.

[40] Collins GS, Reitsma JB, Altman DG, Moons KGM (2015). Transparent reporting of a multivariable prediction model for individual prognosis or diagnosis (TRIPOD): the TRIPOD statement. *Annals of Internal Medicine*, 162(1):55–63.

[41] Pisoni RL, Zepel L, Port FK, Robinson BM (2015). Trends in US vascular access use, patient preferences, and related practices: an update from the US DOPPS practice monitor with international comparisons. *American Journal of Kidney Diseases*, 65(6):905–915.

[42] Bishop CM (2006). *Pattern Recognition and Machine Learning*. Springer, New York.

[43] Troyanskaya O, Cantor M, Sherlock G, et al. (2001). Missing value estimation methods for DNA microarrays. *Bioinformatics*, 17(6):520–525.

[44] Hanley JA, McNeil BJ (1982). The meaning and use of the area under a receiver operating characteristic (ROC) curve. *Radiology*, 143(1):29–36.

[45] Kidney Disease: Improving Global Outcomes (KDIGO) CKD-MBD Update Work Group (2017). KDIGO 2017 clinical practice guideline update for the diagnosis, evaluation, prevention, and treatment of chronic kidney disease–mineral and bone disorder (CKD-MBD). *Kidney International Supplements*, 7(1):1–59.

Webography

— Django REST Framework documentation — https://www.django-rest-framework.org/

— React documentation — https://react.dev/

— scikit-learn documentation — https://scikit-learn.org/stable/

— Ollama project — https://ollama.ai/

— ChromaDB documentation — https://docs.trychroma.com/

— KDIGO guidelines — https://kdigo.org/guidelines/

— Material-UI documentation — https://mui.com/

— Chart.js documentation — https://www.chartjs.org/

— PostgreSQL 16 documentation — https://www.postgresql.org/docs/16/

— Docker documentation — https://docs.docker.com/